

Software Design Description

Mboa Compiler Construction Lab

Deterministic compiler. Project-scoped evidence. Recoverable private work.

NON-AI / PRIVATE / EVIDENCE-LED

Source-grounded design for the non-AI assignment product. Full-size UML sheets follow the A4 narrative and are also published in the standalone atlas. Software diagrams and successful tests do not replace authentic fieldwork.

AUTHOR GROUP

Kwete Ngouba Junior Rayan	ICTU20241377
Djemtchimo Noukui Bruno Jonatan	ICTU20241585
Amina Boubakary	ICTU20241870

Version 3.0 | 22 September 2026

Assignment due: 29 September 2026

Contents

1	Architecture and Decision Record	3
1.1	Default product	3
1.2	Deployment and trust boundary	3
1.3	Explicit legacy boundary	4
2	Source Map and UML Conventions	4
2.1	Exact class scope	4
3	Accounts, Request Selection and Isolation	5
3.1	One request, two storage adapters	5
4	Private Data Model and Recovery	6
4.1	Screenshot pipeline	6
4.2	Document format and verified restore	7
5	Manual Collection, References and Local Imports	7
5.1	Raw evidence and annotation	7
5.2	Source-labelled learning material	7
5.3	Bounded server-side extraction	7
6	Deterministic Compiler Design	8
6.1	Lexer and report	8
6.2	Grammar and predictive parser	8
6.3	Optimization evidence	9
7	Coursework Evidence and Draft Deliverables	9
8	Frontend Boundaries and Compatibility Views	10
8.1	Why desktop diagrams remain	10
9	Validation, Measurement and Open Gates	11
10	Full-size UML Plates	12

List of Figures

1	Current non-AI components and request-local private storage	13
2	Durable single-host deployment and retained identity services	14
3	Assignment user goals and identity/recovery use cases	15
4	Account and web-boundary production classes	16
5	Private storage, coursework, legacy history and recovery classes	17
6	Compiler runtime classes and deterministic function modules	18
7	Raw-preserving individual analysis request	19
8	Lexical result DTOs and true nested association	20
9	Manual collection, reference, metadata and health classes	21
10	Local import, coursework protocol and practice classes	22
11	Reduced runtime configuration and function boundaries	23
12	Frontend runtime class, function components and hooks	24
13	Preserved desktop application and audio ownership classes	25
14	Human fieldwork and deterministic coursework analysis loop	26
15	Desktop compatibility: guarded collection save	27
16	Private workspace recovery decisions	28
17	Desktop compatibility: entry/editor lifecycle	29
18	Desktop compatibility: capture and retry lifecycle	30
19	Account access followed by a scoped private mutation	31
20	Optional Google authorization and explicit linking	32
21	Dual DatasetStorage and CourseworkStorage request binding	33
22	Desktop compatibility: staged attachment and truthful save	34
23	Desktop compatibility: search and cached selection	35
24	Desktop compatibility: changed-field editing and review	36
25	Desktop compatibility: confirmed deletion	37
26	Desktop compatibility: microphone and WAV publication	38
27	Desktop compatibility: library and OS playback	39
28	Desktop compatibility: one-snapshot statistics	40
29	Read-only source-labelled reference lookup	41
30	Local extraction and explicit reviewed collection save	42
31	Grammar analysis and category-token parser execution	43
32	Private profile save and saved-evidence draft export	44
33	Genuine screenshot normalization and private byte retrieval	45
34	Verified backup restore with version check and safety copy	46

1 Architecture and Decision Record

1.1 Default product

Figure 1 maps the implemented components. `backend.main.create_app` defaults to authenticated operation with compiler/coursework routes enabled. FastAPI serves the built React application and API on one origin. The product reuses its actual lexer, grammar analysis, predictive parser, corpus analysis and draft export; it does not replace those features with a generic dashboard.

Generation modules, keys/models/timeouts/quotas and translation/chat/suggestion/ explanation endpoints are removed. Health returns `status=ok` and `mode=compiler`. Google OAuth and SMTP are retained identity/account-mail facilities. The `httpx` dependency remains needed by Google code/key exchange; its presence is not evidence of an active AI provider.

1. **Evidence before conclusions.** Raw manual text remains distinct from glosses, references and constructed examples. The default CFG is illustrative until the group justifies it from real statements.
2. **Deterministic language processing.** Regex/lexicon rules classify tokens; explicit CFG algorithms produce transformations, sets/table and traces. No provider determines acceptance or writes the group's discussion.
3. **Identity before storage.** A verified session supplies the server-owned account directory. Project selectors are validated within that account, never treated as arbitrary filesystem paths.
4. **Dual request-local binding.** Dataset and coursework dispatch through separate protocols backed by the same selected `WorkspaceStore`. No global CSV/profile path is reassigned for a web request.
5. **Explicit persistence.** Previewing, inspecting examples or running analysis does not silently save or approve field statements. Screenshots and profile changes require deliberate actions.
6. **Recoverable compatibility.** Existing accounts, recordings and legacy saved histories remain. Workspace format 2 adds coursework/screenshots, while format-1 restoration remains supported.

1.2 Deployment and trust boundary

Figure 2 shows one durable application host behind an HTTPS reverse proxy. Private SQLite/audio/backup paths are outside public static assets and restricted to the operator/service account. Local development uses loopback and, by default, a private email outbox. Production requires HTTPS, real SMTP, secure cookies, trusted hosts/proxy headers and mandatory authentication. Vite is an optional development server, not the production web host.

The application is not a stateless multi-host database service and account isolation is not end-to-end encryption. Off-host recovery must preserve identity data and protected configuration as well as workspace data. A per-account ZIP does not recreate authentication identities.

Text extraction and compiler execution occur on the application server. There is no generative provider, OCR, remote transcription or browser dictation integration. Optional browser read-aloud is not evidence processing. The UI selects only voices that the browser reports as `localService`, and refuses read-aloud when none is suitable rather than choosing a remote fallback. Voice availability/locality reporting belongs to the browser/OS and is not an application-level guarantee of every underlying network behavior.

1.3 Explicit legacy boundary

The desktop collector remains production source, so its real classes and selected workflows are still documented. Its CSV/audio path is a compatibility workflow, not the default authenticated product. Unbound coursework functions can access legacy profile/screenshot files only in the explicit no-auth maintenance context. The factory rejects `require_auth=False` in production. New private accounts never adopt the old CSV, author identities or screenshots.

2 Source Map and UML Conventions

Source area	Implemented responsibility
backend/main.py, web.py, auth.py	Route composition, built assets, health, request bounds/headers and verified account/session/Google/email boundary
backend/workspaces.py, workspace_backups.py	Account/project SQLite, revisions, legacy history, dual request binding, format-2 document validation and verified restore
backend/coursework*.py	Profile and screenshot protocol, assignment status, lexical/corpus results, grammar/parser routes and draft deliverable export
backend/import*.py, dictionary.py, examples.py	Bounded local extraction; source-labelled reference and constructed practice lookup without real-corpus mutation
compiler/lexer, compiler/parser	Regex classification, approved word map, phrases, formal grammar transforms, FIRST/FOLLOW, predictive table and bounded parser traces
data_collector/dataset.py	DatasetStorage protocol dispatch, shared entry semantics and legacy CSV fallback
data_collector/App.py, audio_utils.py	Optional desktop GUI, local recorder and guarded audio/file lifecycle
frontend/src	Authenticated function components, compiler/collection UI, local draft state, typed API boundary and request cancellation

2.1 Exact class scope

The checked inventory `docs/diagrams/class-coverage.json` enumerates all authored production Python/TypeScript classes. It includes private classes, protocols and the lexer `Token` generated by `namedtuple`. It excludes tests, maintenance tools, external framework classes and erased TypeScript interfaces.

Function modules, hooks, grammar dictionaries, table rows, screenshots and parse steps are not promoted into invented runtime classes. Selected methods are actual source operations; DTO serialization/validation inherited from Pydantic is identified as inherited behavior. The frontend's authored `ApiError` is distinct from its function components.

- Hollow triangles point to base types; dotted realization arrows point to structural protocol

contracts, not nominal Python inheritance.

- True nested/stored references have multiplicities. Inheritance, realization and module dependencies do not receive decorative cardinalities. Reusable DTO references are not exclusive-lifecycle compositions.
- Every displayed sequence communication has an active sender, an explicit receiver activation and a matching reply/deactivation. Actors are active while participating; this does not claim CPU execution by people.
- State diagrams describe actual logical GUI/recorder states, not nonexistent enums. Activity decisions have exclusive guards; human assignment tasks are labelled.
- Full-size atlas images are embedded without squeezing complex graphs onto A4. Figures and their source filenames remain traceable.

3 Accounts, Request Selection and Isolation

Figure 4 documents `AuthSettings`, `AuthStore`, identity/input/error models and web-boundary classes. The store retains one settings reference; it returns identity values from SQLite rather than maintaining a fictional aggregate of live user DTOs. Passwords use Argon2id and session/verification/reset values are stored as digests.

Figure 19 separates registration, email verification, sign-in and a later authenticated mutation. Private routes require a valid session; mutations additionally require allowed Origin and session-specific CSRF. Request throttles remain, but generation quotas do not. Blocking identity/storage work is dispatched off the event loop.

Optional Google authorization uses PKCE, browser-bound state, nonce and signed identity-token validation (Figure 20). Invalid/replayed state cannot authorize an exchange. A token-exchange failure does not trigger the signing-key request. Equal email alone does not link an existing password account; signed-in explicit linking is required. Logs describe safe failure stages without leaking codes, tokens, credentials or provider response bodies.

3.1 One request, two storage adapters

Figure 21 makes the key isolation decision explicit:

1. Middleware resolves `UserIdentity` and validates the requested project within that user's workspace.
2. The context factory initializes `WorkspaceStore` in a worker thread and binds the workspace `ContextVar`.
3. `dataset.use_storage(workspace)` and `coursework_store.use_storage(workspace)` bind both adapters.
4. Routes call normal dataset/coursework functions; protocol dispatch uses the selected account/project store.
5. Context tokens are reset on exit, including exceptions. Invalid selection does not fall back to global legacy files.

Native screenshot/audio URLs carry a project query parameter because they cannot always attach the application's project header. The server still derives the account solely from the cookie, not from that query.

4 Private Data Model and Recovery

Figure 5 lists the workspace input, history and backup classes. It also shows structural conformance to two contracts: `DatasetStorage` and `CourseworkStorage`. The store retains a file-lock object, its paths and the selected project ID. Database rows are not retained Pydantic object associations.

Table / resource	Record structure	Scope / behavior
projects	ID, name, creation time	Account-wide; General is retained
entries	ID, project ID, entry JSON	Raw wording, glosses, notes, provenance, annotations and optional audio reference
revisions	ID, project/entry IDs, action/time, before/after JSON	Recorded changes; version-checked restore
history	ID, project ID, kind/title, content JSON, timestamps	Legacy translations/conversations remain readable/exportable
coursework	Project ID primary key, profile JSON	At most one saved profile per project
screenshots	Hex ID, project ID, name, base64 PNG content	At most six images per project; byte reads require ID and project
meta	Key/value, including workspace version	New profile and screenshot mutations bump version
audio / backups	Private generated files and validated ZIPs	Account storage/size limits; no public static exposure

Saving a profile validates its grammar before upserting JSON. A missing profile returns a new default at the coursework layer, with three blank member strings and the illustrative grammar; it is not automatically persisted or marked genuine. Screenshot/profile records also prevent deletion of a supposedly empty project. No original account or private data migration/deletion is part of this change.

4.1 Screenshot pipeline

Figure 33 follows actual upload and byte retrieval. The route allows a 3 MiB JSON request body to carry base64; the generic JSON cap remains 1 MiB. The request model bounds the name/data URL. The store validates base64, PNG/JPEG content, a 2 MiB input and 12-megapixel resolution, then converts to PNG and again rejects a normalized result over 2 MiB. Storage validates that PNG before inserting it under an account lock and transaction.

The returned URL includes the selected project. Reads query both image ID and project ID in that authenticated account's SQLite database, return image bytes with private/no-store and nosniff headers, and do not reveal a legacy filesystem path. Deletion has the same project condition and increments the version. Image decoding cannot establish that the student supplied a genuine capture.

4.2 Document format and verified restore

Workspace document **format 2** includes **coursework** and **screenshots** arrays. Validation checks project references, unique IDs, record bounds, profile schema/grammar and decoded PNG data. A legacy **format-1 workspace document** is copied and normalized to format 2 with empty coursework/screenshots. The enclosing ZIP **manifest remains format 1**. These version numbers describe different layers.

Figures 34 and 16 show preview, integrity/schema/media validation, expected-version checking and the pre-restore safety snapshot. A profile/image mutation after preview invalidates that version. Media is staged without overwriting colliding committed files; logical rows are replaced transactionally after validation. Failed staging/commit removes only newly staged files. A post-commit cleanup failure is explicit and does not mean the committed database automatically rolled back.

Checksums detect changed bytes, not trusted authorship. Backups preserve all private projects, coursework, screenshots and recordings but not identity secrets or server settings. They are separate from the coursework deliverable bundle.

5 Manual Collection, References and Local Imports

5.1 Raw evidence and annotation

Figure 9 separates editable entry inputs, stored-entry DTOs, read-only dictionary values, meta-data/health and explicit errors. Raw **StoredText** fields preserve observed spelling/whitespace and reject blank-only content. Stored glosses and notes also retain their exact input. The existing **ShortText** validation trims leading and trailing whitespace from supplied contributor and source-location values on create/edit. Import previews and frontend handoffs preserve those provenance values before this save-time normalization. Omitted patch fields and existing records are not rewritten by that rule.

ID, timestamp and attachment operations are not freely editable request fields. Duplicate entries and stale IDs are errors rather than silent replacement. Evidence changes invalidate approval unless explicitly reviewed as the new evidence.

The assignment target counts Sentence entries, not Word/Phrase annotation rows. Corpus analysis can inspect all saved entries; this does not inflate the 10–15-statement target. Suggested topic gaps are for discussion and remain review items, not a mandatory one-statement-in-every-topic gate.

5.2 Source-labelled learning material

Figure 29 loads validated versioned references and returns paginated search results with file-name/line and competing senses. The 179 source rows are not 179 unique words or manually collected statements. English meanings and etymological origins do not supply reviewed French glosses or part-of-speech labels.

The 26 supplied practice statements are explicitly constructed. **PracticeEntry.constructed** is always true. Reading either source never inserts it into the empirical corpus. Authenticity remains the responsibility of the group, not a side effect of a search result or parse success.

5.3 Bounded server-side extraction

Figure 10 contains **ImportEntry** and **ImportPreview**; the latter is local-only. No suggestion request/result models or provider choices remain. Figure 30 separates file preview from an explicit

reviewed save.

The upload must contain exactly one file and no cloud-consent fields. Supported formats are UTF-8 TXT/Markdown/CSV/JSON, DOCX and text-layer PDF. Local extraction is bounded to 12 MiB input, 40,000 characters, 40 PDF pages and 100 structured drafts. DOCX ZIP/XML and PDF safety checks apply before returning text. Unsupported media, encrypted/invalid documents and image-only PDFs are rejected, with no OCR/transcription fallback.

Partly scanned PDFs may return readable text with warnings; students must compare the complete source. Structured imports keep allowed raw/provenance fields without trimming contributor/location values, but cannot import approval. Only the later collection save applies short-metadata normalization; statement text, glosses and notes remain exact. Preview does not retain the original file as an attachment or save entries automatically.

6 Deterministic Compiler Design

6.1 Lexer and report

Figure 6 identifies real runtime classes and function modules. The generated `Token` holds raw text and category; API `TokenResult` values are shown in Figure 8. The individual `AnalyzeRequest` preserves text and rejects blanks (Figure 7).

The lexer splits raw tokens with regex, applies structural number/punctuation rules, then considers explicitly approved, unambiguous language-labelled Word annotations, small base lexical lists, morphological heuristics and UNKNOWN. Sentence rows are not training data. Conflicting annotations cannot silently choose one category. Multiword verb/slang patterns and code-mixed transitions are separate annotations, not claims of a complete multilingual model. Structural and multiword regexes are precompiled. Matching respects token boundaries, preserves raw spelling and returns phrase occurrences in source order; multiword annotations do not collapse the token stream. For example, the brief’s illustrative *drop me* phrase remains separate underlying tokens, rather than being replaced by one VERB terminal to force parser acceptance. Such an illustration is not a collected field statement.

Corpus reporting computes raw statements/tokens, token and category frequencies, unknown counts and normalized spelling-variation candidates. Case/accents are used for comparison without rewriting original wording. Variation grouping is orthographic, not proof of shared meaning; `_LIKE` categories are heuristic guesses, not certified POS.

6.2 Grammar and predictive parser

Figure 31 traces the shared corpus and single-input path. Grammar notation is read and validated, supported direct/indirect left recursion is removed, common prefixes are factored, and each changed stage retains bounded before/after evidence. FIRST/FOLLOW are computed to a fixed point and the LL(1) table records conflicts rather than silently picking one candidate.

The parser consumes category labels, not a probabilistic sentence meaning. A bounded stack machine records pre-action stack/input/action traces, matches the end marker and reports accepted/error/consumed results. Residual recursion, conflicting tables, invalid grammar and excess work are explicit boundaries. LL(1) is the implemented alternative; there is no LR/SLR implementation claim.

One corpus analysis reads a bounded selected-project snapshot (at most 500 entries and 100,000 characters), analyzes the submitted grammar and parses each entry. Single-input parsing uses the same selected project’s reviewed word map. The coursework operation holds the same reen-

trant account workspace lock used by mutations across profile/corpus/screenshot reads and analysis. Cooperating account mutations therefore cannot mix evidence revisions within the operation. Analysis does not implicitly save the editor/profile.

Interpretation boundary

A grammar can accept a constructed sentence or reject genuine speech. Neither result establishes authenticity or linguistic correctness. The default CFG remains an illustration until the group relates its rules to real observations and explains coverage, ambiguity and rejection.

6.3 Optimization evidence

Only the public illustrative starter grammar (including its saved stripped form) is cached as immutable JSON. Each caller receives newly decoded mutable results. Custom grammar text/comments, corpus statements and reviewed word annotations are not retained by that grammar cache.

Algorithmic changes must preserve deterministic outputs, trace semantics and caller isolation. Any timing claim belongs with its actual fixture, warm-up, sample size, environment and exclusions.

The source-hash-verified record `compiler/output/benchmark_verified.json`, recorded 22 September 2026 at 09:46:18 UTC on Windows 11 / AMD64 / CPython 3.14.6, contains thirteen individual synthetic cases and a separate paired comparison. It uses 21 samples of 150 iterations and reports five warm-ups per case; each sample is a batch-average timing, not individual request latency. The paired batches alternate order in one process and compare the same current grammar preparation algorithm on the exact whitespace-stripped public starter.

Repeated saved-starter preparation	Median per call
Current uncached preparation	1,042.155 μ s
Current cached preparation	207.775 μ s
Ratio of medians	approximately 5.02×

This is a narrow cache benefit for repeated public-starter preparation, **not** a speedup claim for custom grammars, every compiler algorithm, end-to-end HTTP/UI response time or linguistic quality. Older before/after captures were different-time runs with substantial timing noise and include unchanged or slower cases; they remain historical comparisons, not proof that all operations improved. No account/project data was used in the synthetic benchmark. The current record's eleven source hashes were checked against the implementation before documenting these values.

7 Coursework Evidence and Draft Deliverables

Figure 14 places human collection and interpretation around implemented analysis steps. Three distinct nonblank names, authentic manual-transcription confirmation, collection method, grammar rationale and discussion are explicit profile data, not hidden generated content.

Figure 32 separates saving a submitted profile snapshot from exporting saved evidence. The frontend must not mark later edits saved when an older request completes. Unsaved profile/screenshot work is finished before the explicit export. The backend uses saved profile/corpus/images and current deterministic analysis to assemble:

- A **DRAFT** printable HTML report with 25 sections and missing-input markers, not a certified 25-page finished report.

- Editable PowerPoint slides and timing notes totaling ten minutes: three per member and one shared minute.
- Raw dataset, token, frequency, variation and acceptance CSV; grammar, lexical specification, parser cases, profile and checklist JSON.
- Genuine supplied screenshot PNGs, relevant source snapshots and own-corpus regression cases.

The coursework ZIP does **not** contain raw recordings. Full workspace backups preserve recordings together with coursework. The shared reentrant account lock protects profile data, corpus rows and screenshot reads throughout export and analysis. This provides a coherent saved-evidence snapshot against cooperating workspace mutations. This is not a transaction over developer-edited source files or arbitrary external filesystem writers. Source snapshots omit environment secrets but raw text/contributor metadata are included, so the archive must be shared only with authorized course recipients.

Generated test expectations snapshot the parser's current decisions; they are not independent proof that a linguistic judgment is right. The group must review accepted/rejected cases, author original discussion, inspect genuine screenshots and verify actual 25–30-page output, capped at 30. The SRS/SDD PDFs are separate engineering documents and have no such report-page target.

8 Frontend Boundaries and Compatibility Views

Figure 12 keeps React function components/hooks distinct from the authored `ApiError` class. The API boundary attaches cookie, CSRF/project selection and cancellation context; older identity/project responses cannot overwrite a new session. Project switching/sign-out warns before discarding dirty drafts. Local recording and optional read-aloud are separate capabilities; neither is browser dictation.

Legacy history can be inspected/exported without sending data to removed translation/chat endpoints. Retaining its schemas preserves old account data, not a live generation feature. Figure 11 shows the reduced configuration class and actual function modules; no invented provider service remains in the inventory.

8.1 Why desktop diagrams remain

The optional desktop collector is still authored production code. Its actual `App`, `Tooltip`, `Recorder` and errors remain in Figure 13, with real selected operations and ownership. It must therefore remain covered even though the authenticated compiler is the default product. These diagrams are explicitly labelled *desktop compatibility*, never the current account/project storage flow.

View	Preserved behavior, not a web claim
Figures 15, 22	Ready-draft validation, staged audio, locked CSV save, truthful commit versus refresh/cleanup status
Figures 23, 28	One read snapshot, cached selection, read-only filtering and consistent statistics; failed reads preserve visible data
Figures 24, 25, 17	Changed-field edits, review invalidation, explicit stale-ID errors and confirmed deletion without removing committed audio

View	Preserved behavior, not a web claim
Figures 26, 27, 18	Device capture/stop/WAV-write retry states and library/OS playback fallback; accepted playback is not proof that sound was heard

9 Validation, Measurement and Open Gates

The existing local PlantUML/LaTeX build renders all current sources and compiles atlas, SRS and SDD. The strict checker verifies exact production-class inventory, mapped declarations, one source/image/sheet per diagram, correct SDD/atlas page mapping, balanced call/reply activations, embedded image fingerprints and clean layout/reference logs. It does not prove every UML semantic claim, so class relationships and diagram scope also require source inspection.

Documentation-specific tests exercise missing/obsolete classes, runtime TypeScript versus erased interfaces, generated namedtuple classes, pixel normalization and invalid sequence activations. Final counts/results are recorded in `docs/README.md`; no prior suite totals are reused.

Backend/compiler tests and isolated browser fixtures establish software behavior, not genuine fieldwork or public deployment acceptance. Historical `docs/evidence/*.json` remains unmodified. Prior Gemini, translation, assistant, dictation and cloud-media acceptance has no bearing on the removed architecture.

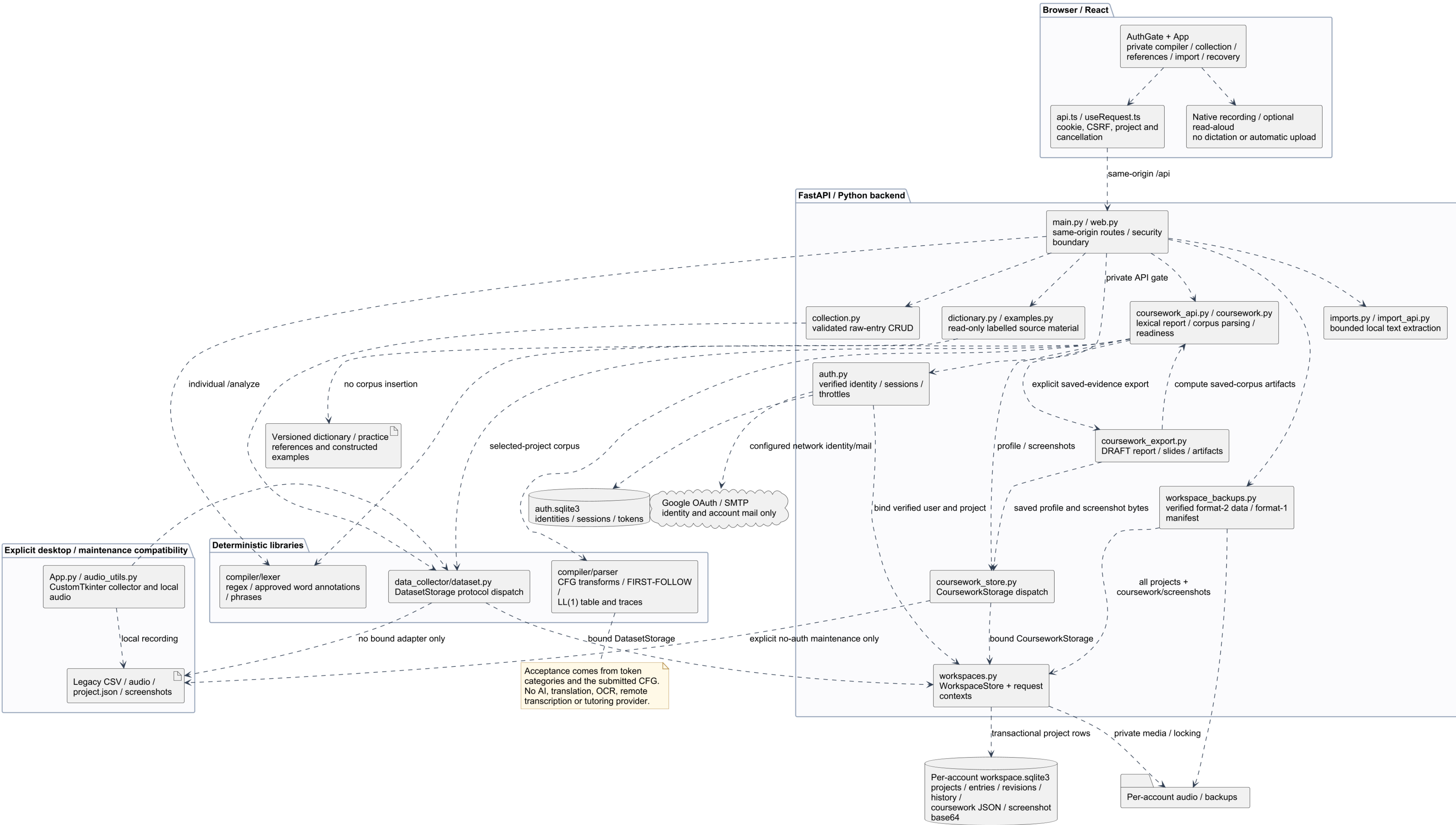
Open coursework gates

The reviewed corpus was empty. Genuine manual text, corpus-derived grammar reasoning, authentic screenshots, original linguistic discussion, final pagination and rehearsed delivery remain outstanding. No diagram, generated draft or synthetic test is presented as their substitute.

10 Full-size UML Plates

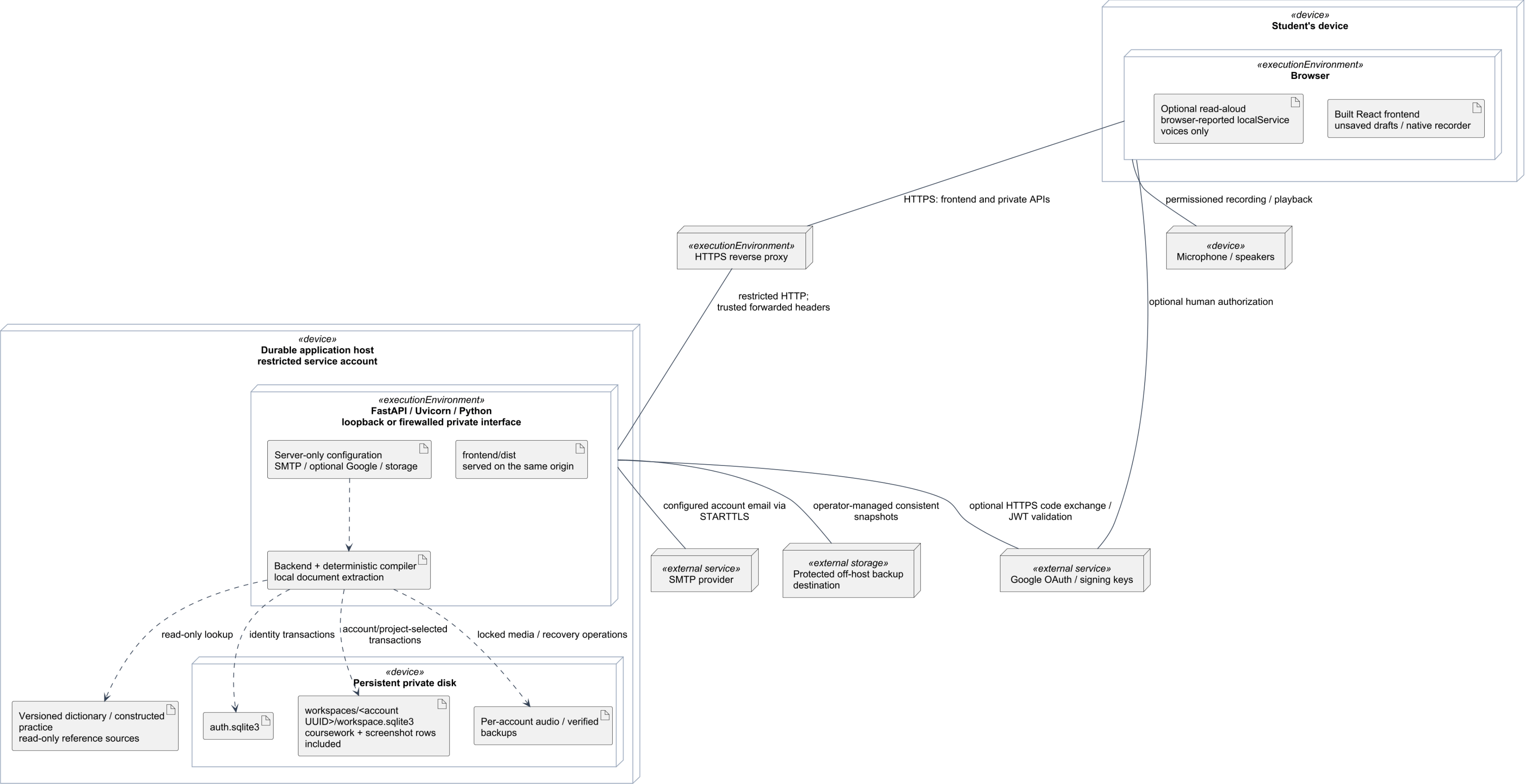
The following pages are the exact images in the standalone atlas. They use their own sheet sizes for legibility; zoom or tiled printing is preferable to forcing every detailed graph onto a single A4 page.

Non-AI compiler workspace: authenticated request-local storage



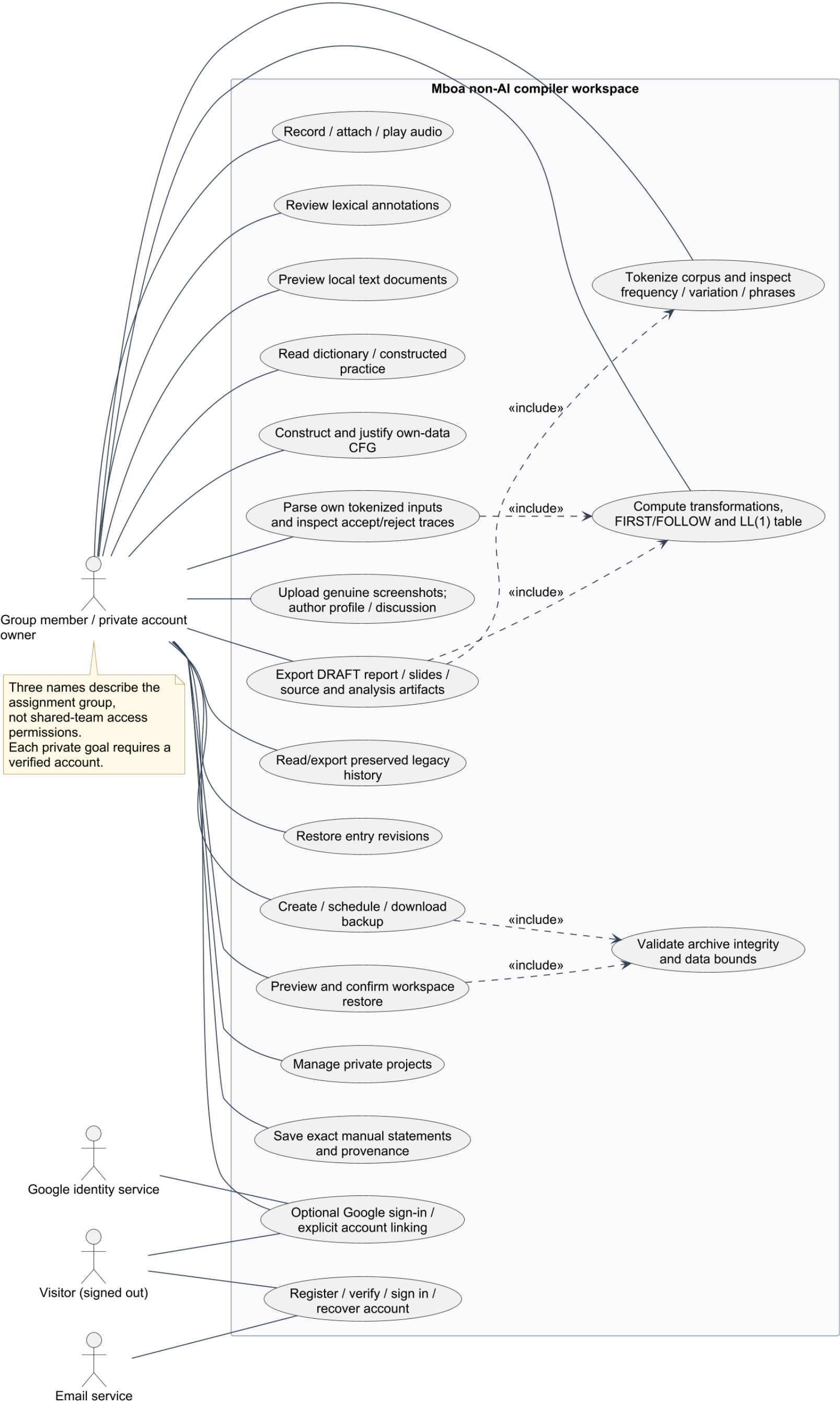
Dashed arrows are dependencies, not associations with invented multiplicities.
Authentication is default; production rejects require_auth=False.
Legacy files are never assigned to a private account automatically.
Coursework ZIPs contain draft deliverables; workspace backups retain audio.
TTS selects only browser-reported localService voices, without remote fallback.
Browser/OS reporting is not an application-level guarantee of all voice/network behavior.

Deployment: one durable host, local compiler, retained identity services



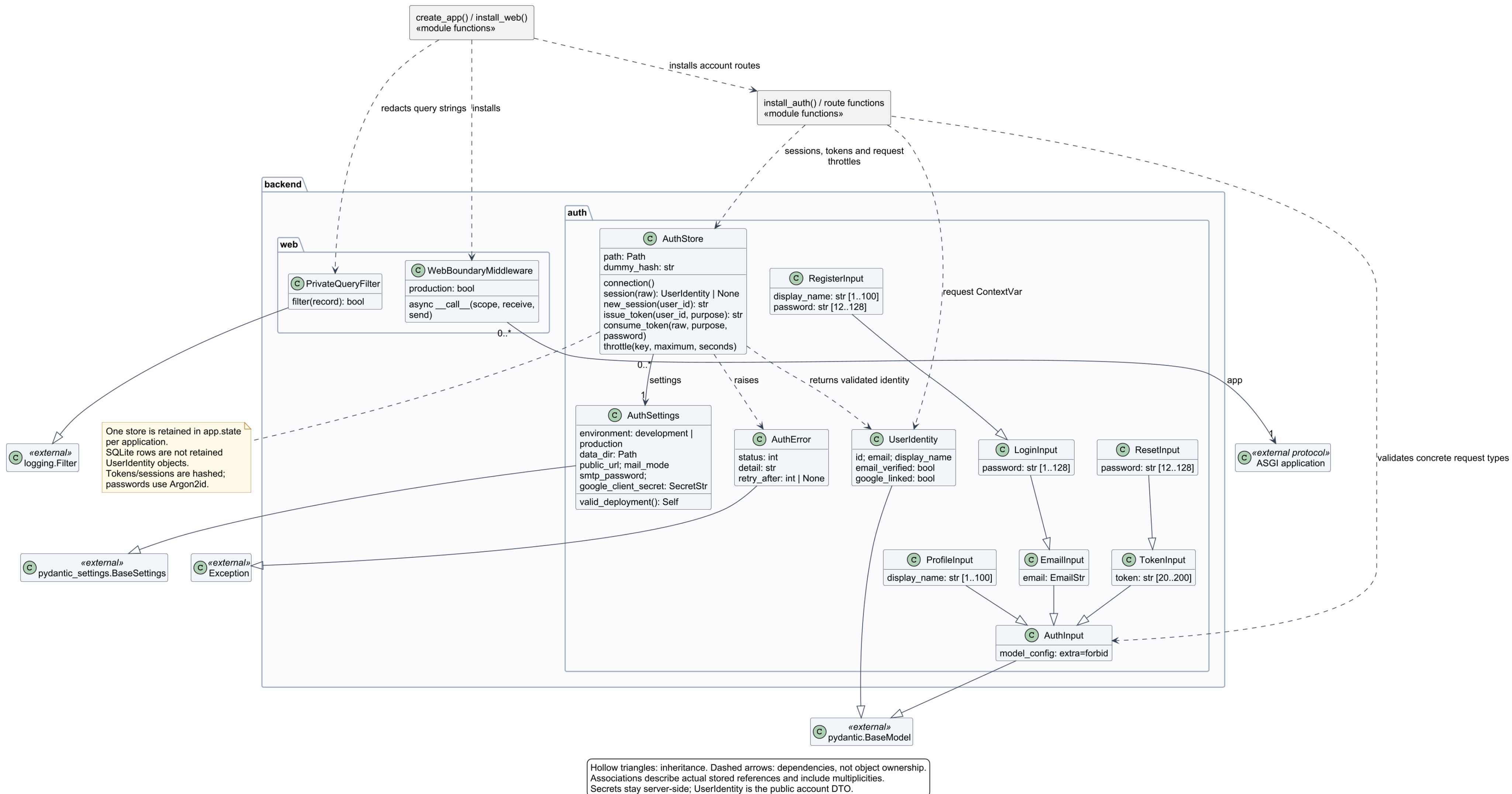
No language-model, OCR or transcription integration exists.
TTS has no remote-voice fallback; the app relies on browser/OS localService reporting.
That reporting is not a guarantee about every underlying browser/OS network behavior.
Google/SMTP are network identity services; local compilation is not a claim of an offline website.
Local development: browser -> loopback FastAPI; no public proxy needed.
Authentication stays enabled; legacy CSV/profile/screenshots are a separate maintenance boundary.
Backup identity/configuration separately from user ZIPs. Account isolation is not E2E encryption.
Do not deploy private SQLite to ephemeral disks or imply a multi-host database cluster.

Assignment user goals and retained identity/recovery

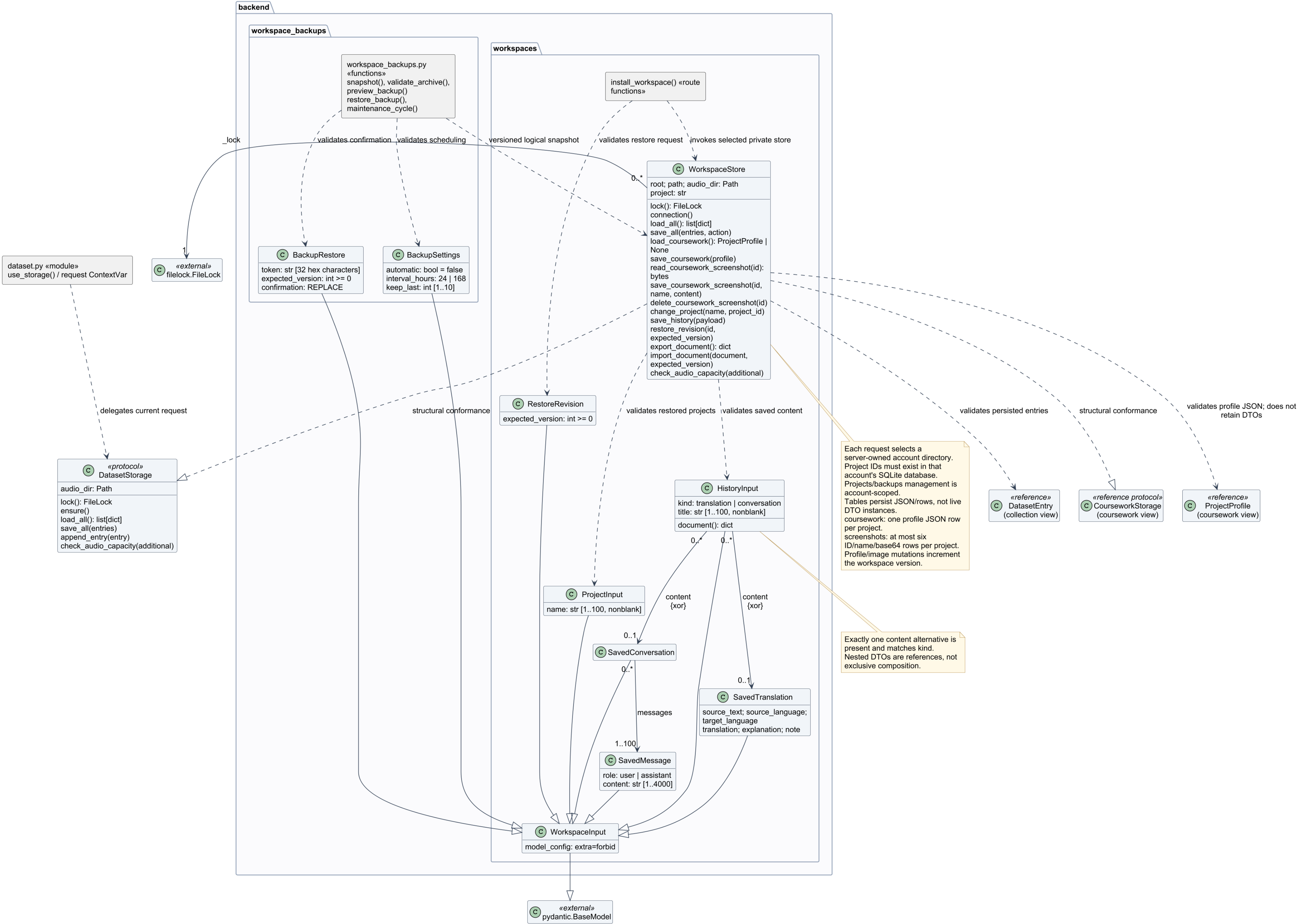


Include points to mandatory reused behavior; no dependency is decorated with cardinalities. Import/reference lookup never automatically saves or authenticates a field statement. Real listening/manual transcription, final pagination and presentation rehearsal remain human work. No AI, translation, chat, OCR, remote transcription or browser dictation use case exists.

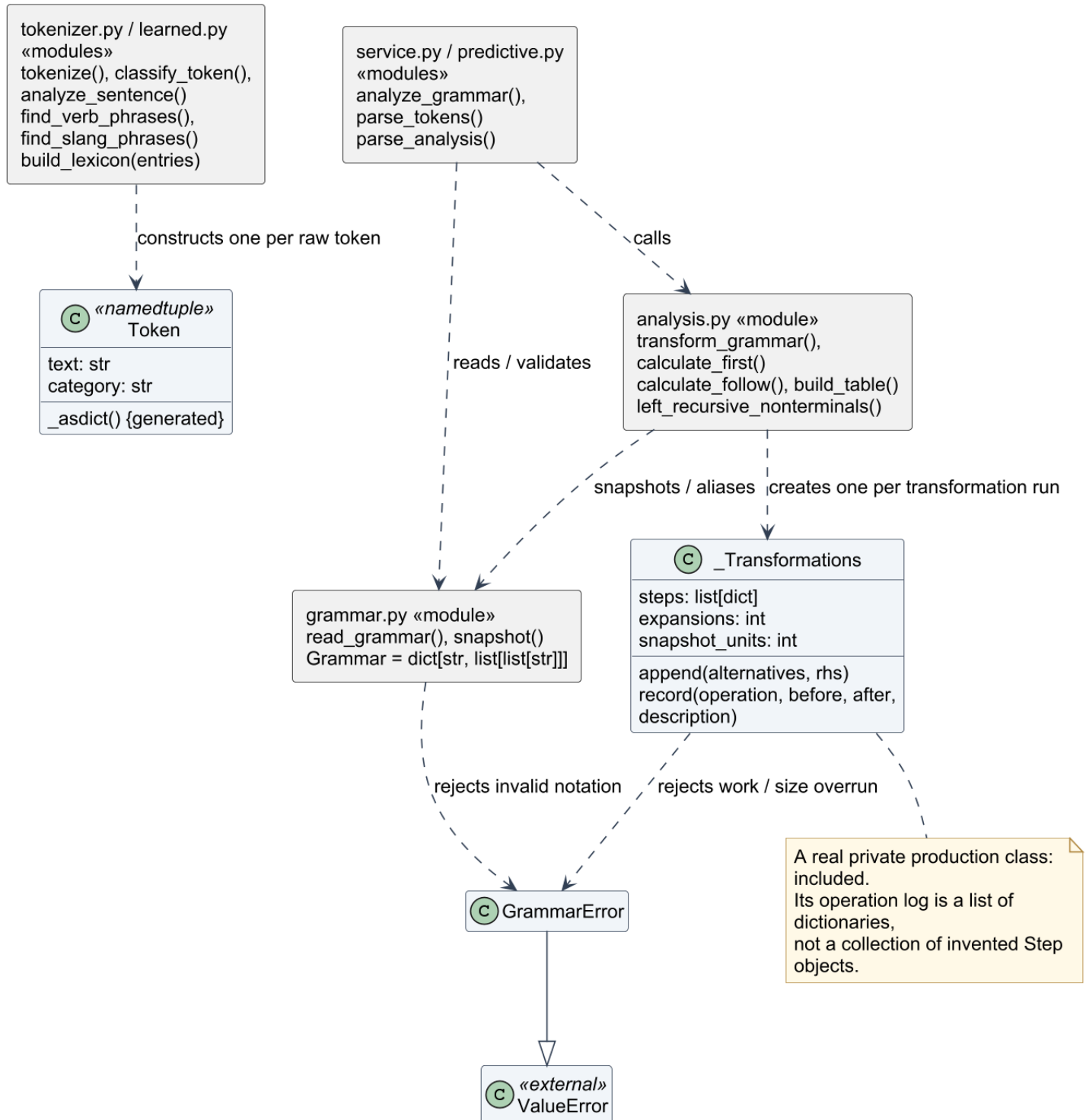
Retained private accounts: actual classes and selected operations



Private projects, coursework, legacy history and recovery



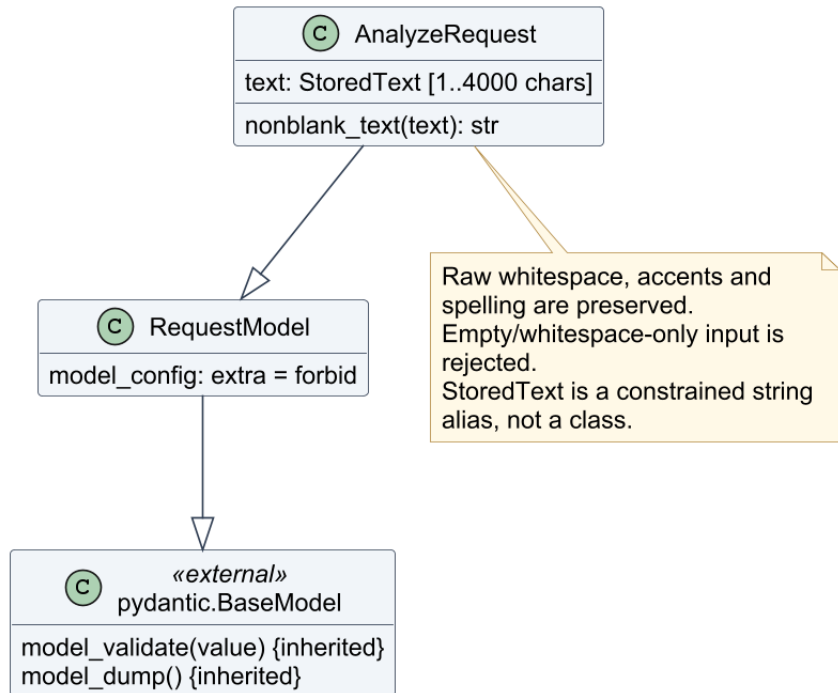
Compiler runtime classes and function modules



A real private production class: included.
 Its operation log is a list of dictionaries, not a collection of invented Step objects.

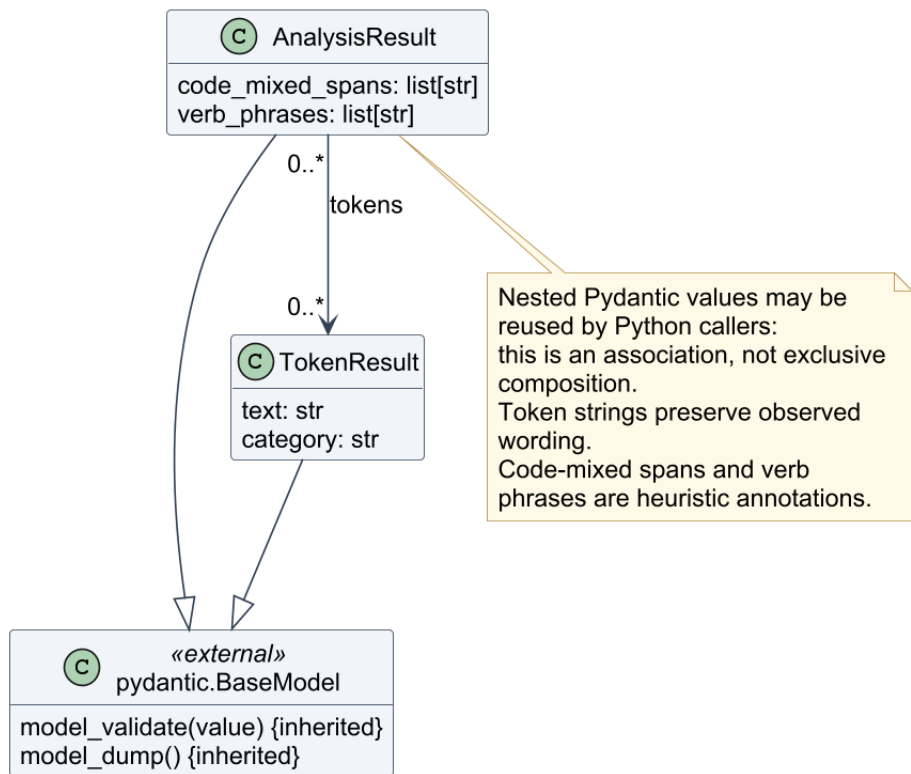
Token is an application-defined runtime class generated by namedtuple.
 Grammar, Production, analysis results and parse traces are aliases/dictionaries.
 There is no LexerService, Grammar class, Parser class or compiler database.
 Lifetimes of function-local values are not drawn as class compositions.
 Structural/phrase regexes are precompiled; raw text and source-ordered phrase matches are preserved.
 Only the public starter grammar is cached as immutable JSON; each result is decoded afresh.
 Custom grammars, corpus text and reviewed annotations are not cached by that service.

Deterministic analysis request contract



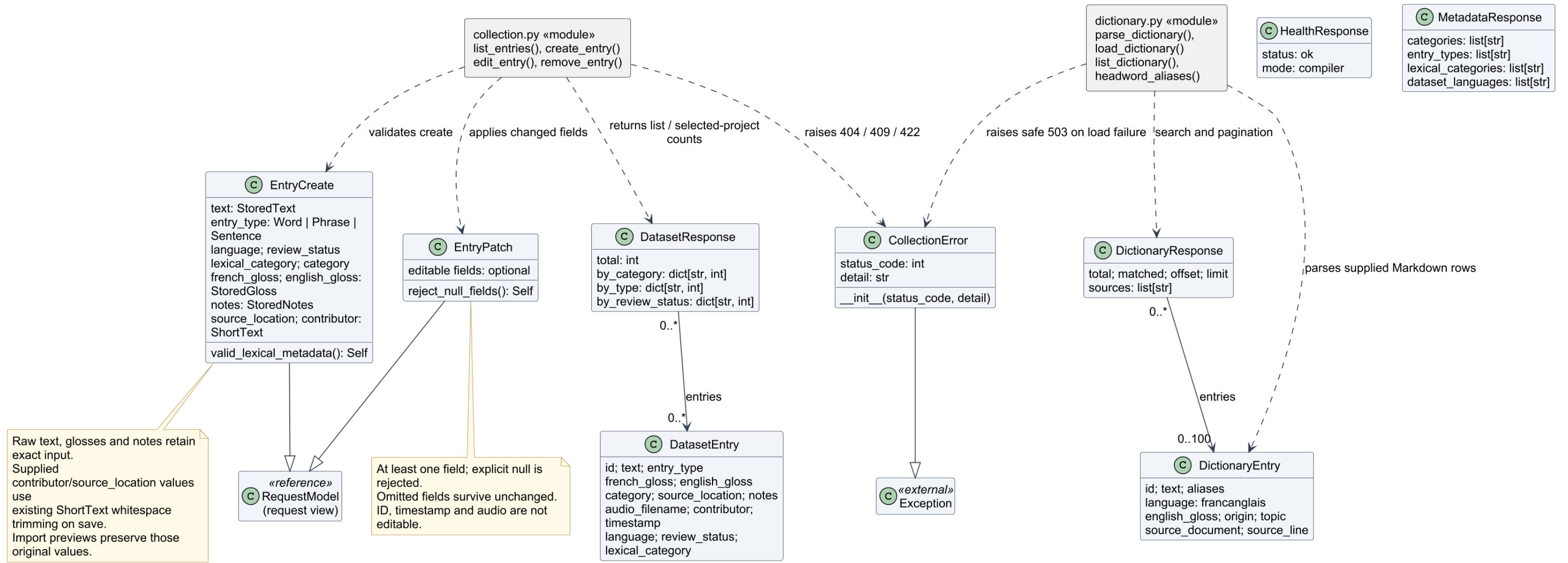
Production classes: backend.schemas.RequestModel and AnalyzeRequest. Concrete collection/coursework request subclasses are in their detail views. Removed translator/chat request classes are not part of the current inventory.

Lexical analysis result values



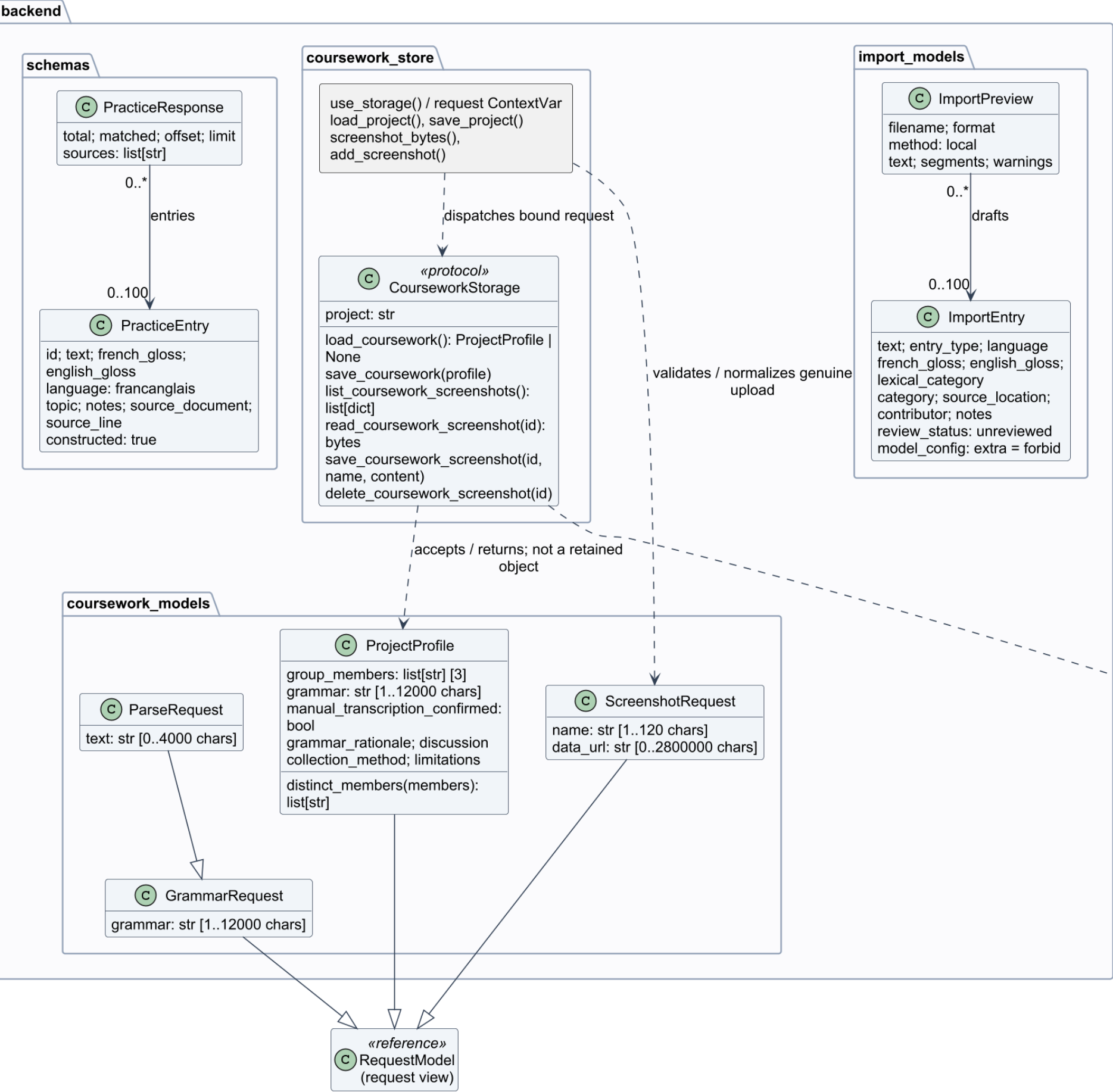
Both primary classes live in `backend.schemas`. Grammar analyses, parse traces and corpus reports are dictionaries, not hidden DTO classes. No translation, provider-evidence or generated-reply classes remain.

Private manual collection and read-only reference DTOs



The six response DTOs directly extend pydantic.BaseModel.
EntryCreate/EntryPatch inherit extra=forbid via RequestModel.
DatasetEntry is an API DTO, not the storage representation:
Authenticated storage serializes dictionaries to private SQLite JSON rows.
Desktop fallback helpers still persist dictionaries as CSV.
Reference entries are read-only, not approved corpus entries.
Their etymological origin does not supply a French gloss or POS.
Pydantic response values inherit model_validate()/model_dump().

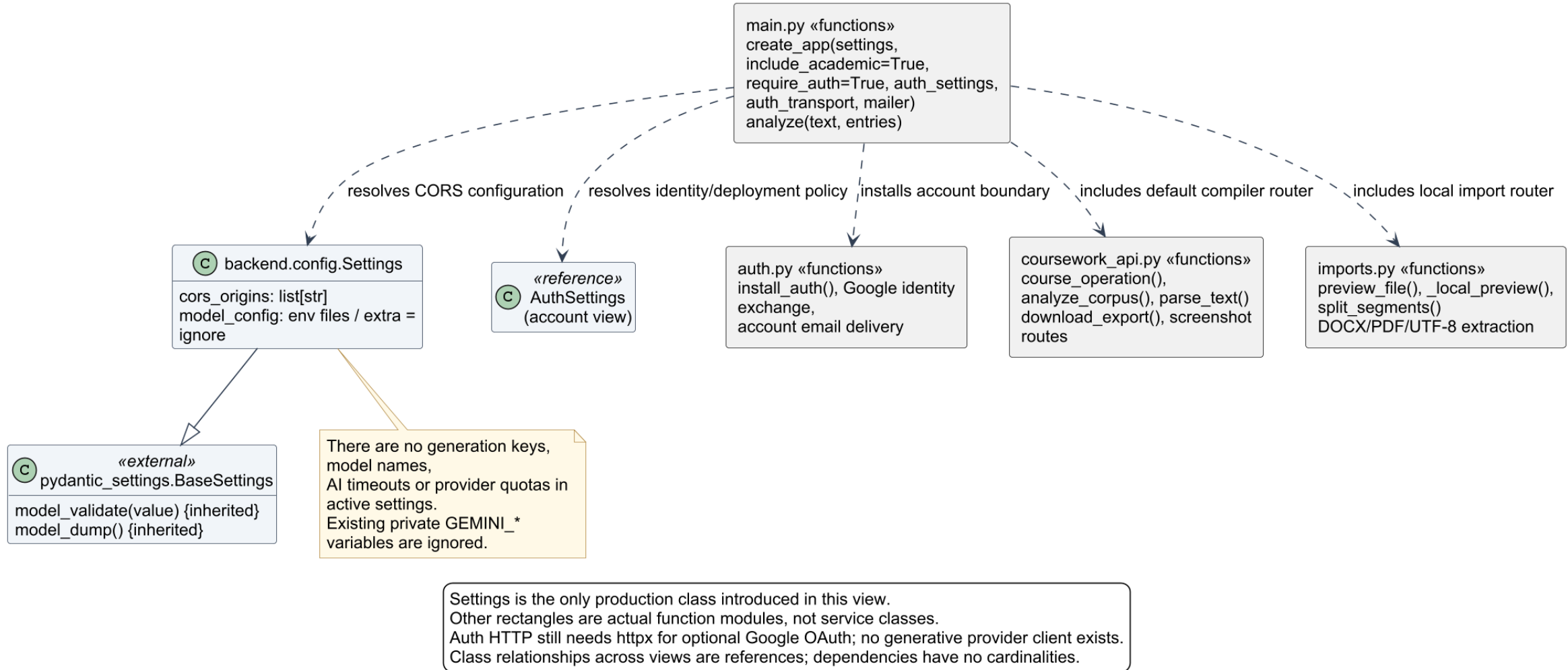
Local imports, private coursework and constructed practice



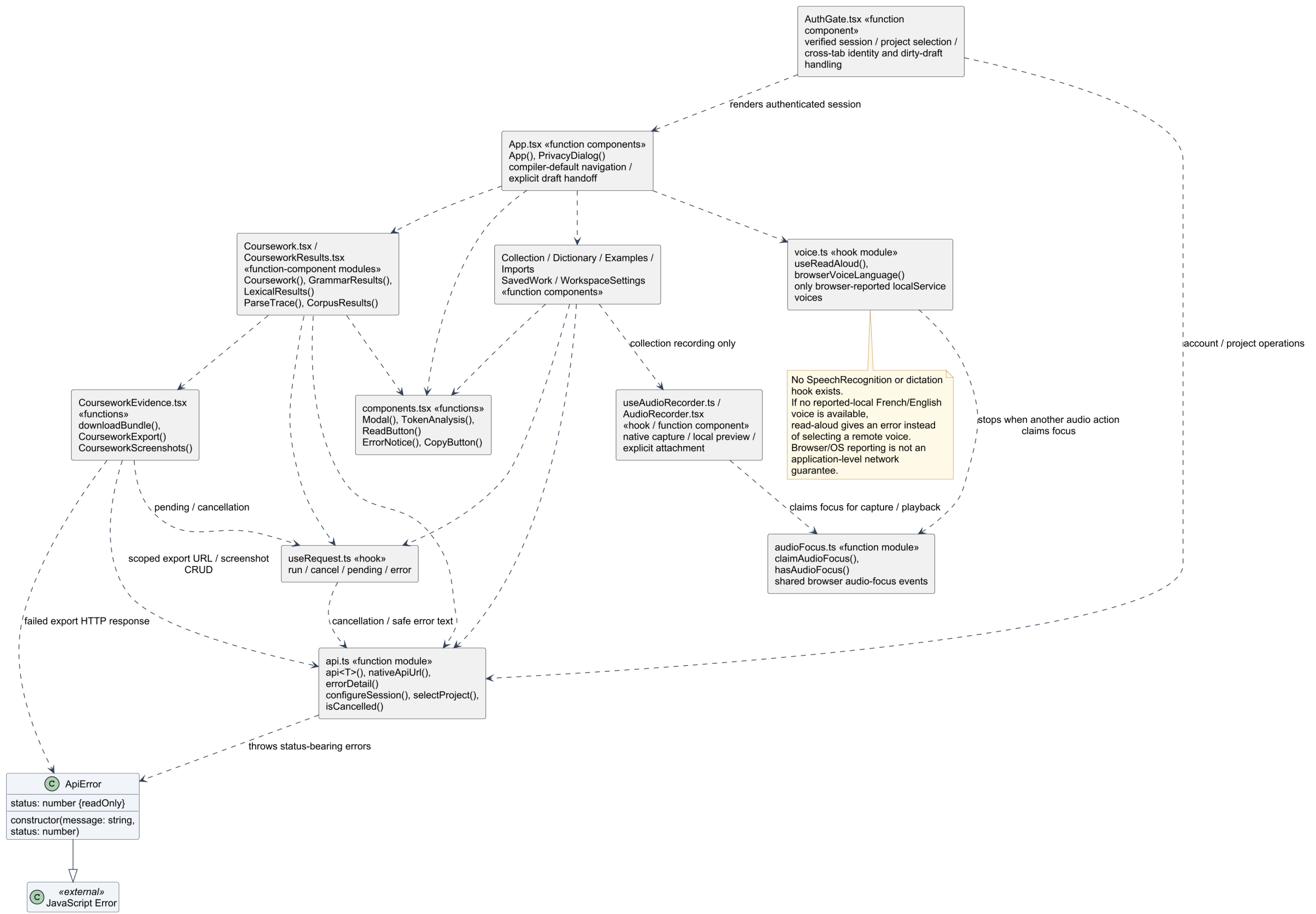
WorkspaceStore structurally implements this protocol. Normal requests never read the legacy global profile/screenshots. Screenshots are rows/bytes, not an invented Screenshot class.

Import/practice DTOs directly extend BaseModel; serialization methods are inherited. Three member strings may be blank initially; nonblank names must be distinct. ImportEntry never accepts an approved status. No suggestion/generation model exists. Preview provenance stays unchanged; only a collection save trims short contributor/location fields. Screenshot limits: 6/project, 2 MiB decoded/normalized, 12 MP; JSON route cap 3 MiB. Reference/practice records do not silently become field statements.

Runtime configuration and actual function boundaries

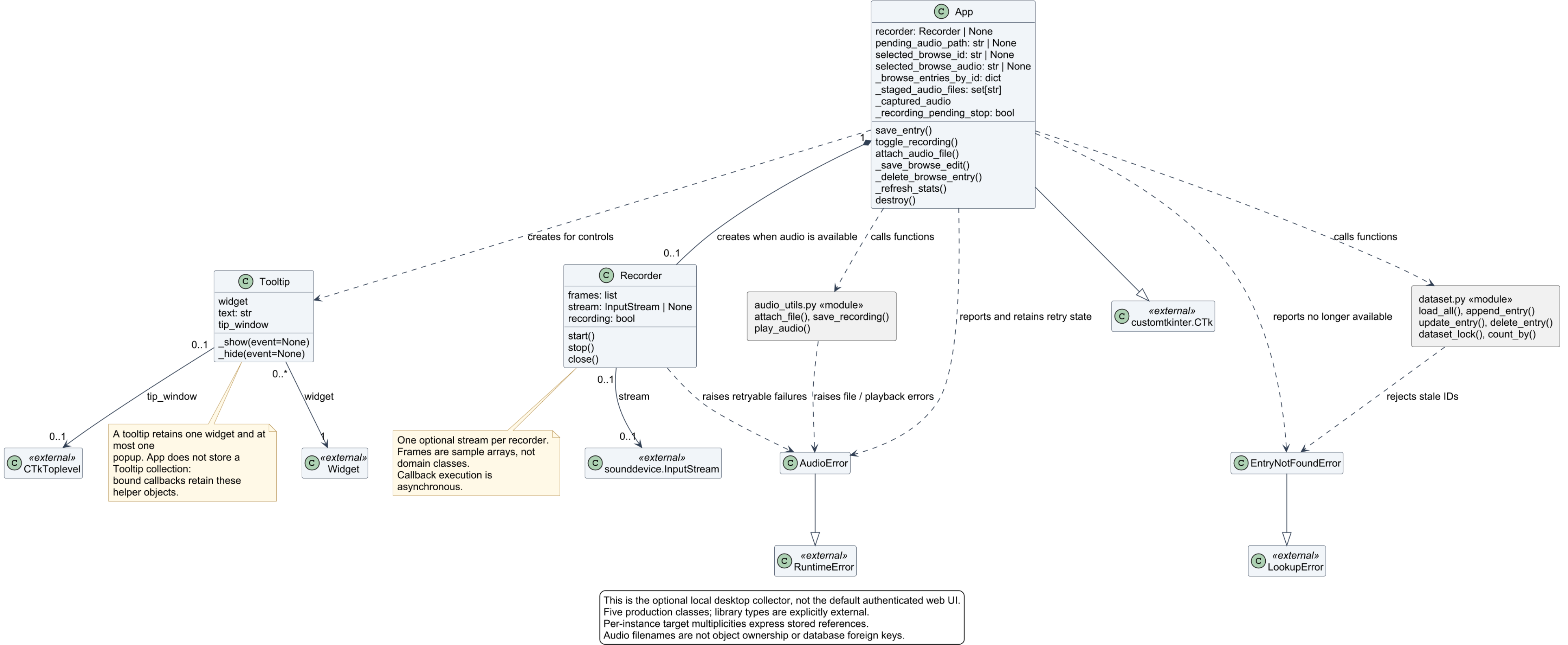


Non-AI frontend: one runtime class, function components and hooks

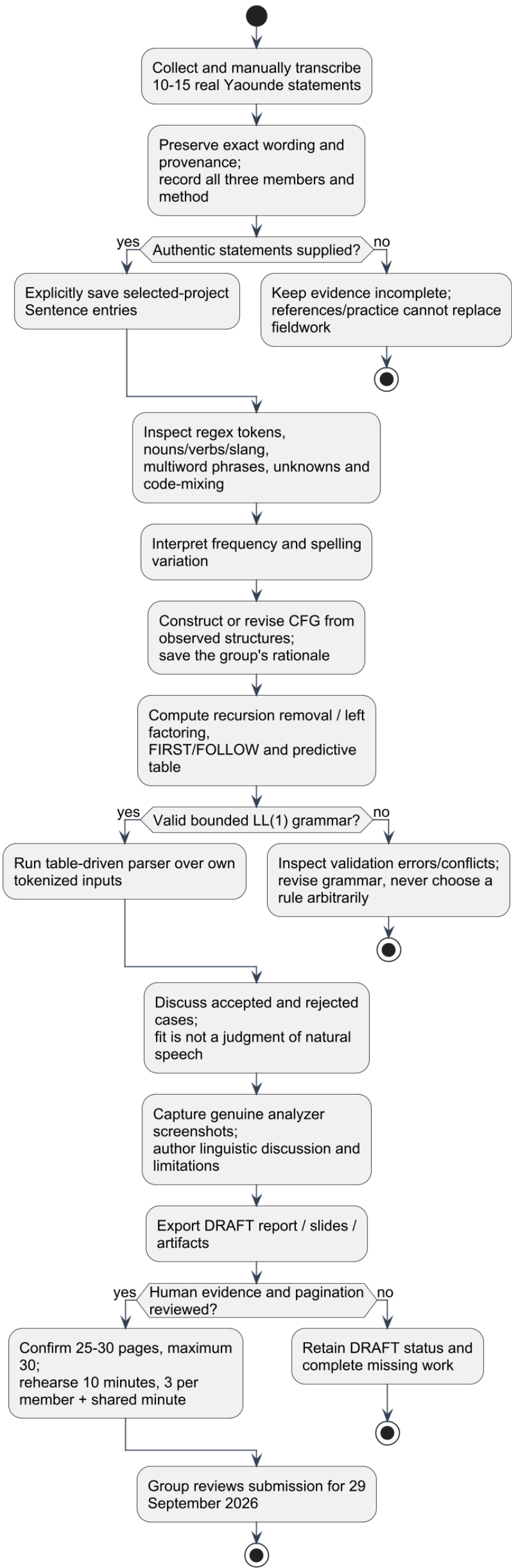


frontend.src.api.ApiError is the authored runtime class.
Interfaces/types, including recorder state shapes, are erased and not invented classes.
Rectangles show dependencies, not object ownership or associations with multiplicities.
Coursework is the default product. History preserves legacy data without generation.
API generation guards prevent older account/project responses replacing current state.

Preserved desktop compatibility: actual classes and audio ownership

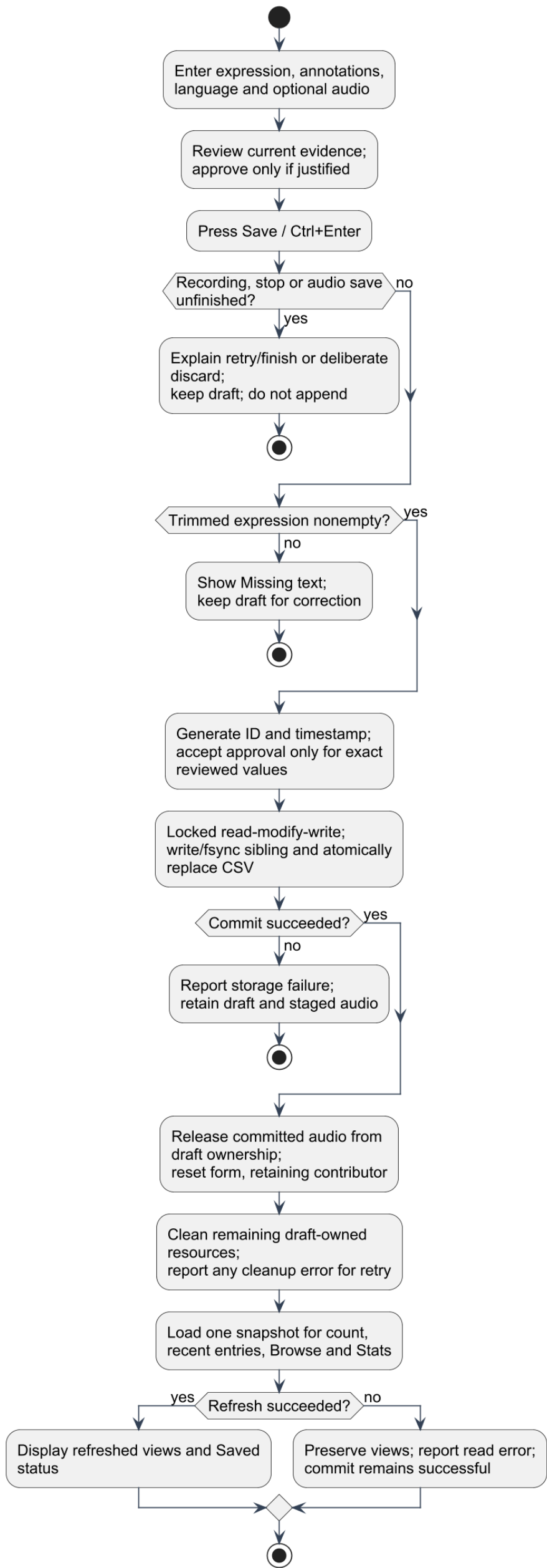


Coursework loop: real observations before formal analysis



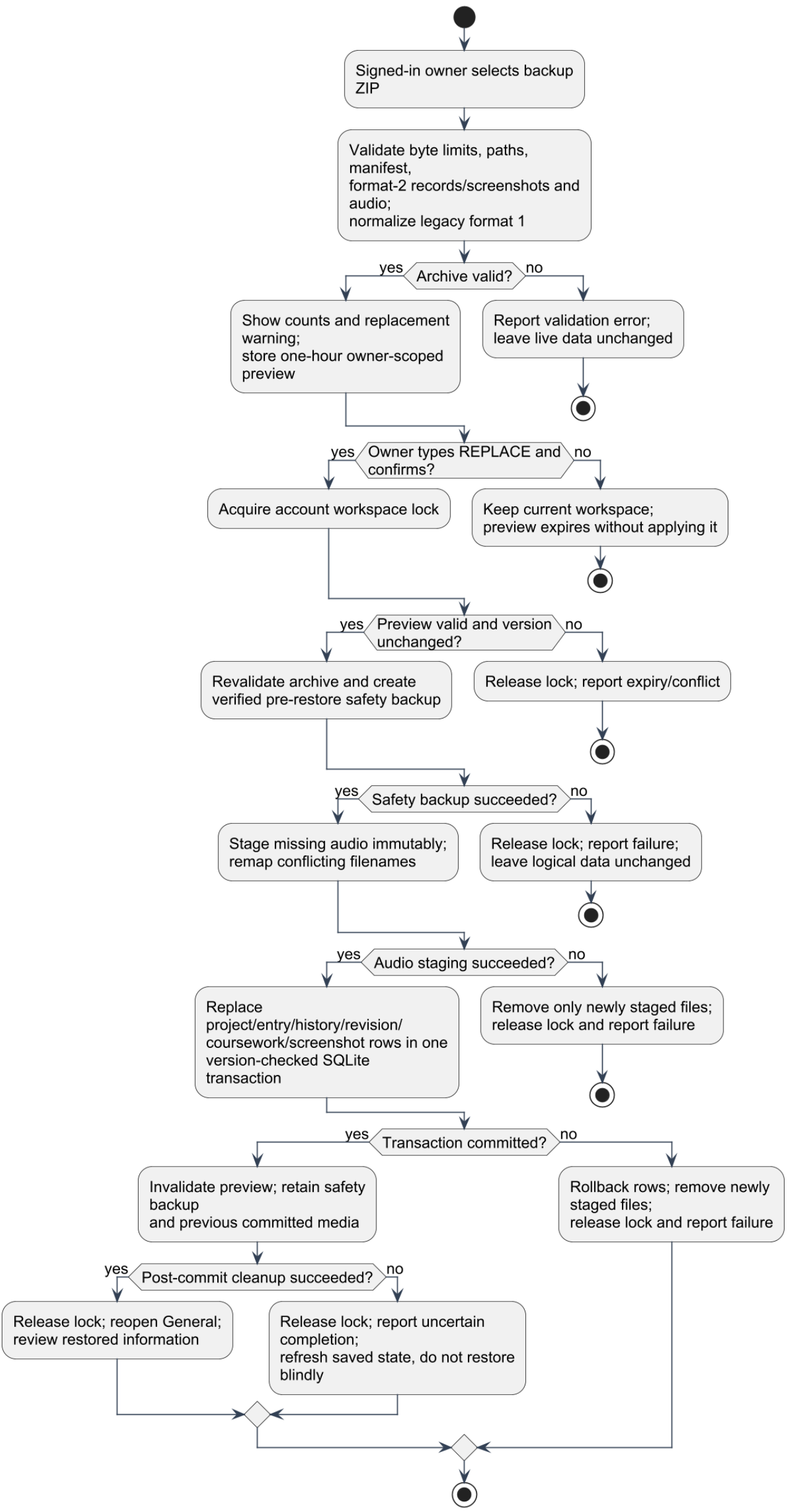
This activity combines implemented software steps and explicitly human coursework tasks. Suggested topic coverage is a discussion aid, not a mandatory one-sentence-per-topic gate. The default CFG is illustrative until justified against real statements. Export is not evidence certification and is separate from the engineering SRS/SDD.

One desktop save attempt, including failure boundaries



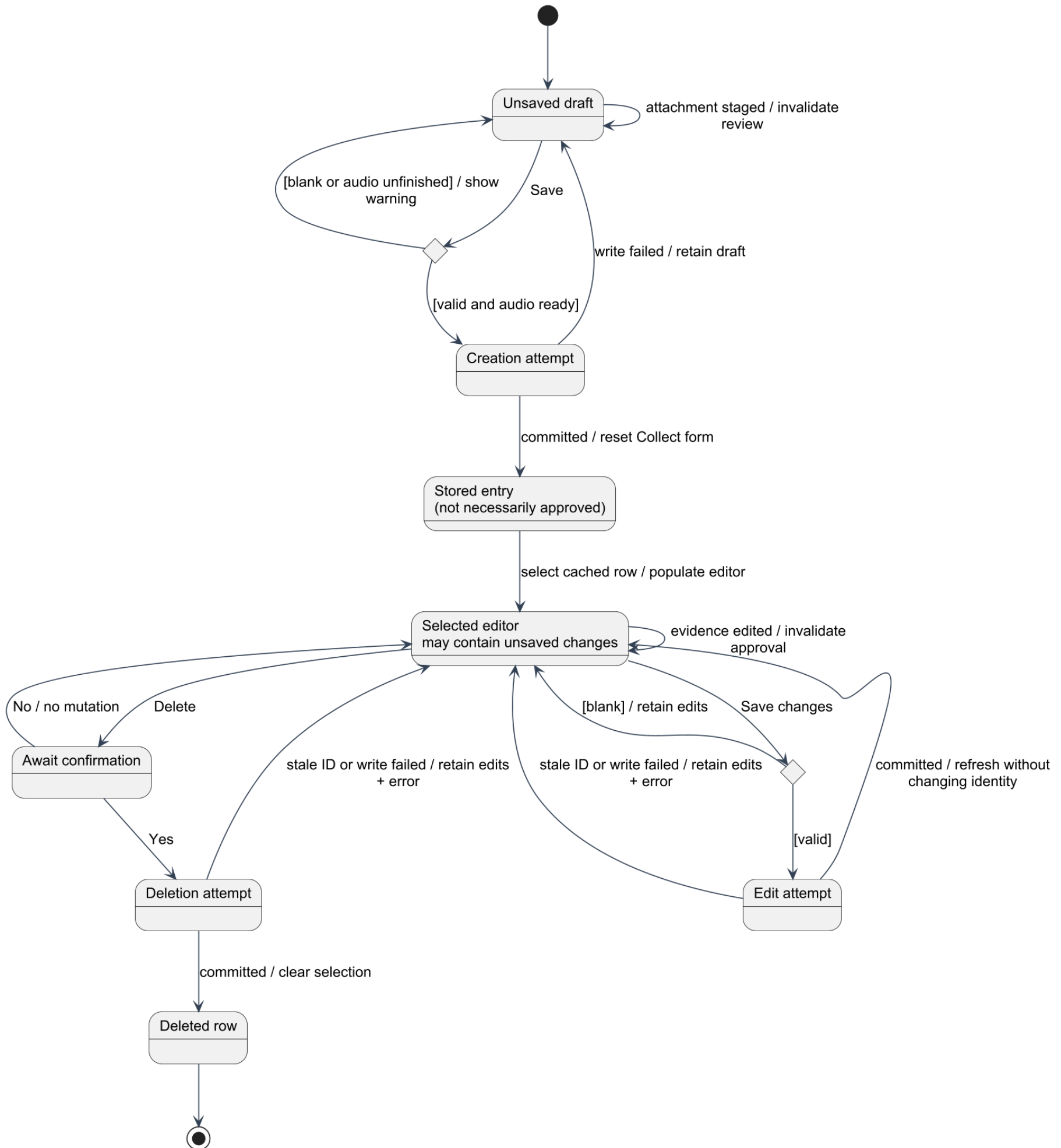
Preserved desktop compatibility only; web saves use private WorkspaceStore transactions. Diamonds represent exclusive guarded decisions, not parallel work. One final node ends this attempt, not the collector application. Duplicate warnings are advisory on desktop; API duplicates return 409. File staging is separate; recording details appear in the sequence view.

Private workspace restore: guarded replacement and failure paths



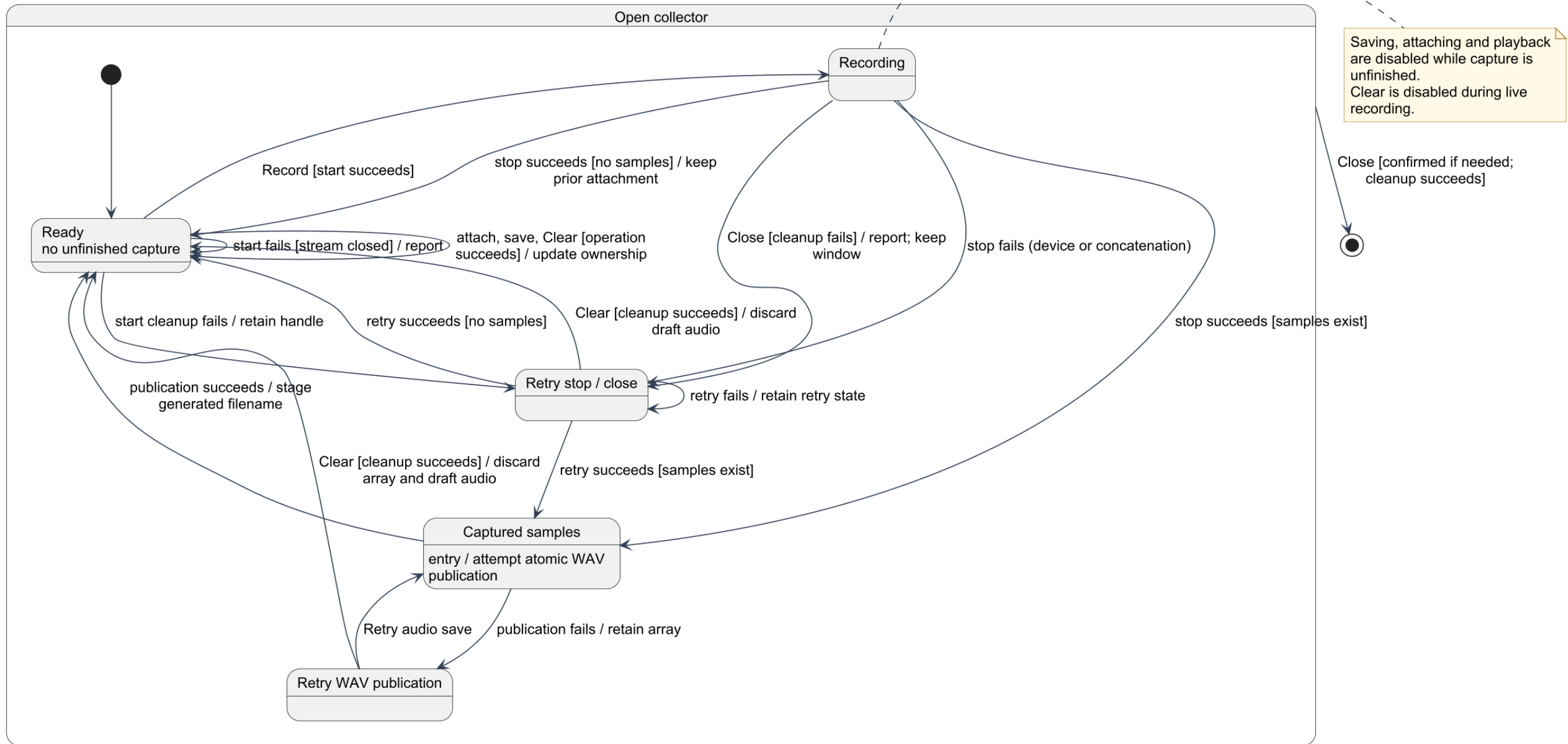
Branch guards are exclusive; each final node ends this attempt.
A cleanup error is explicit and does not imply a committed transaction rolled back.
This activity does not delete account credentials or other users' workspaces.
The workspace document is format 2; the outer archive manifest remains format 1.

Preserved desktop row and selected-editor lifecycle



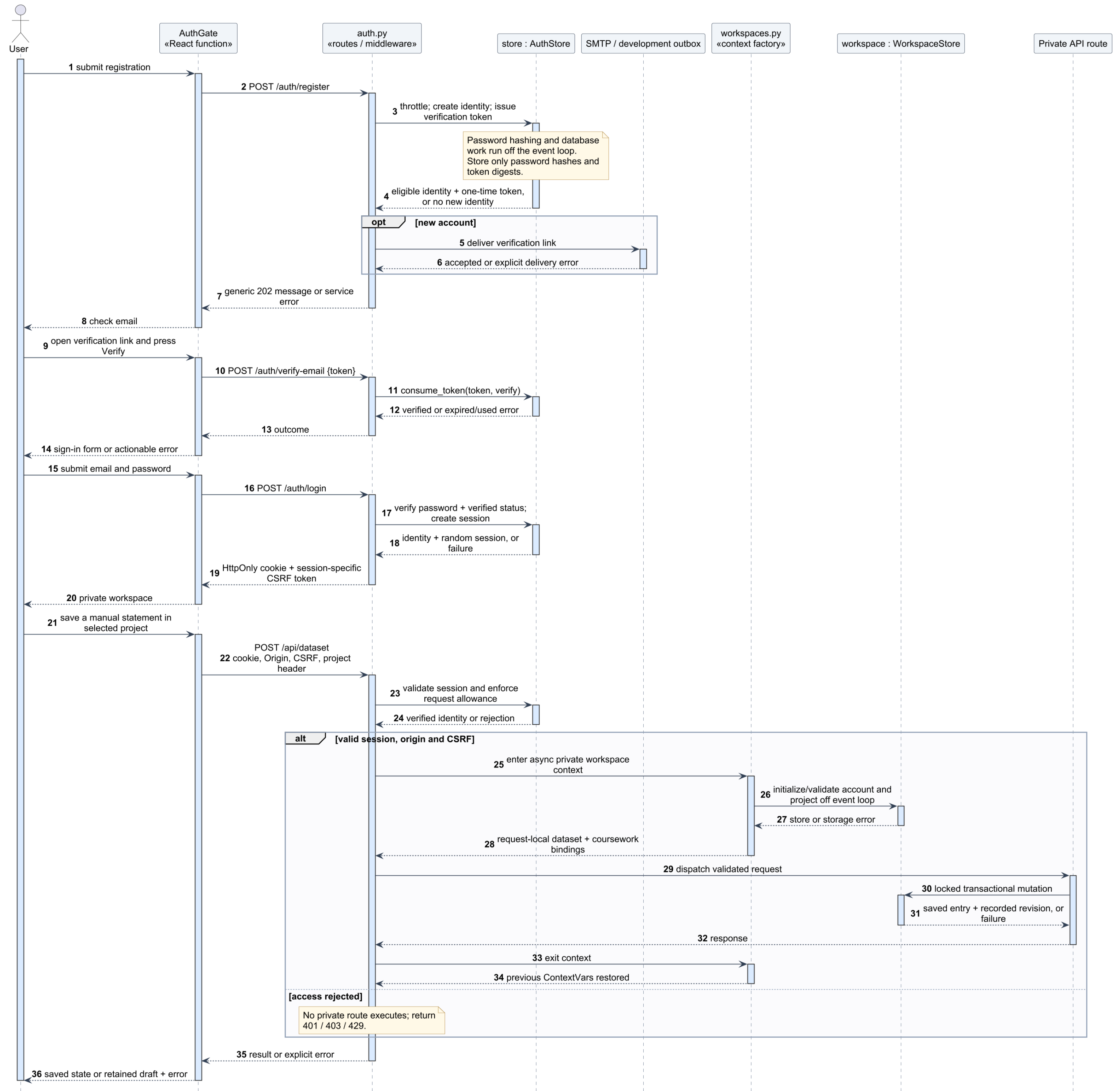
Desktop compatibility view; web account/project lifecycle is separate.
 This is a logical row/editor model, not an implemented enum or whole-app lifecycle.
 The selected editor may outlive a row removed by another writer; stale errors are explicit.
 A successful commit is not reversed by later display/cleanup failure.
 Audio staging has a separate lifetime; deleting a saved row retains its recording.

Desktop capture readiness and retry states

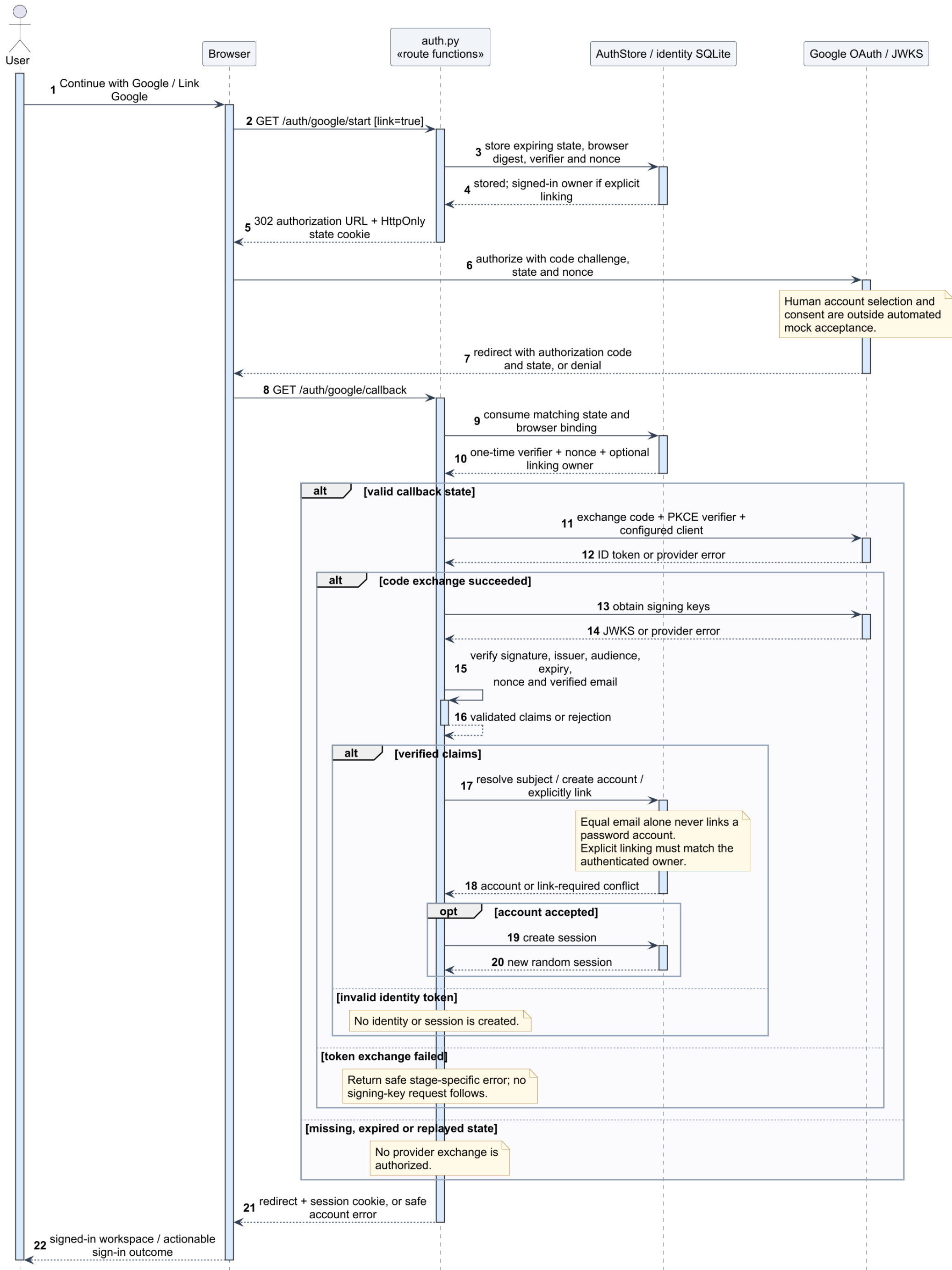


Preserved desktop Recorder/GUI states, not browser dictation or an AI transcription flow. Conceptual states derive from recording, stream, pending-stop and captured-array fields. Ready may contain a staged filename; it does not mean an empty form. Failed cleanup keeps the window/retry state available, not a false closed status. Draft-owned files may be discarded; committed recordings are never owned by the draft.

Email account access and private request binding

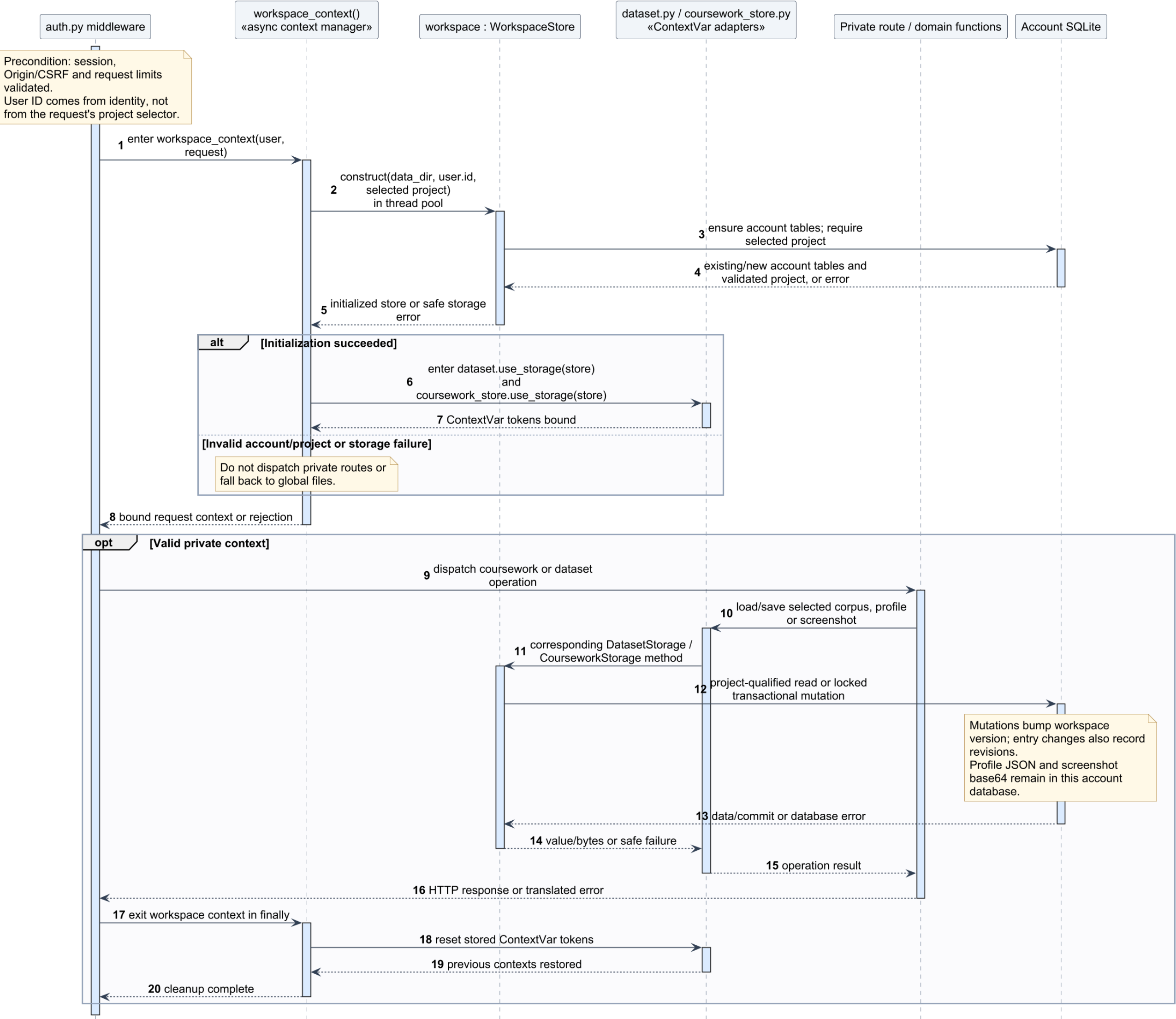


Google sign-in / explicit linking: authorization-code flow with PKCE



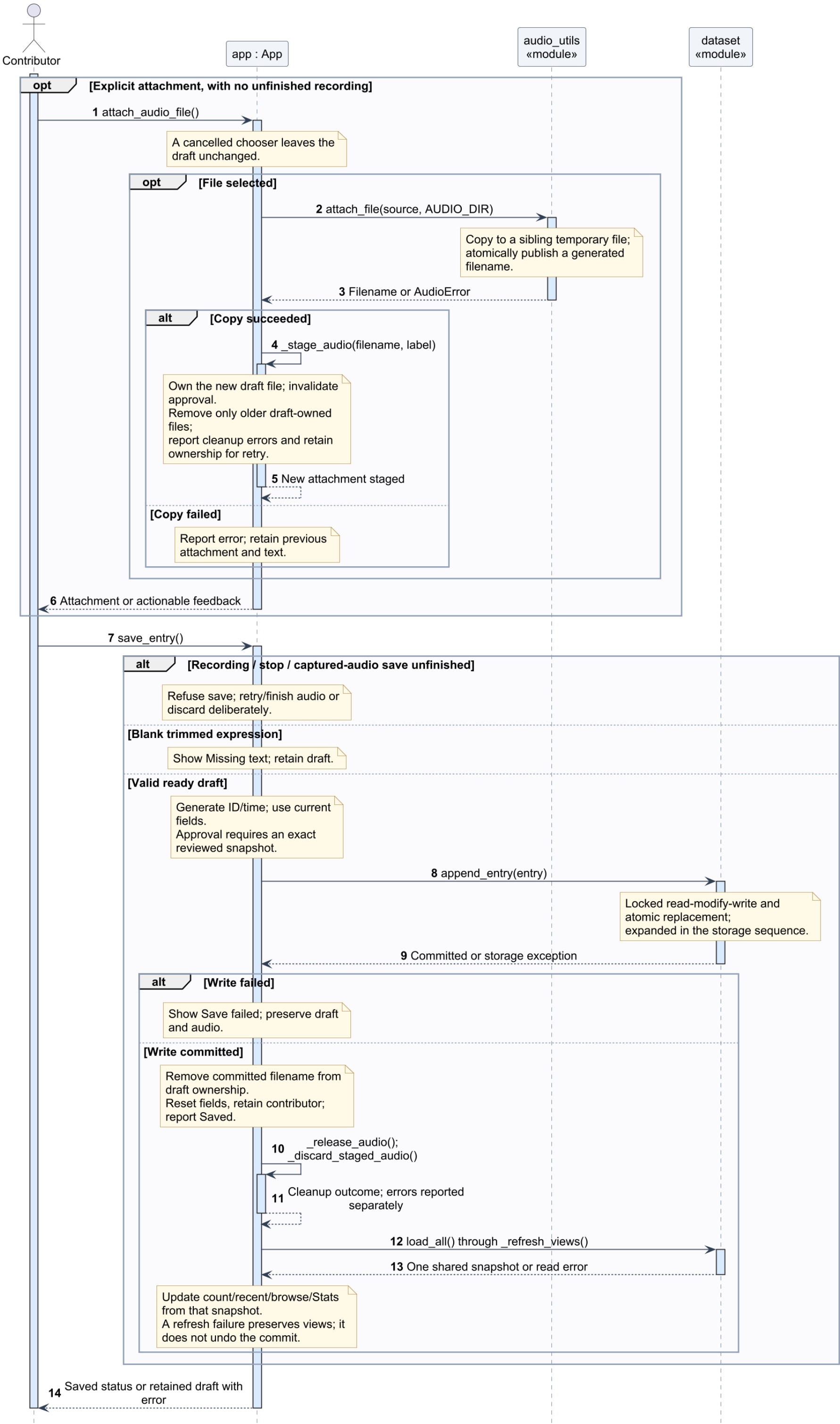
Redirects are HTTP replies; following them begins a new request.
Code, verifier, nonce and credentials are not written to browser storage.
Query values are redacted from application access logs.
This optional identity exchange is retained; it is not a language-provider call.

One request, one account/project: dual storage context binding



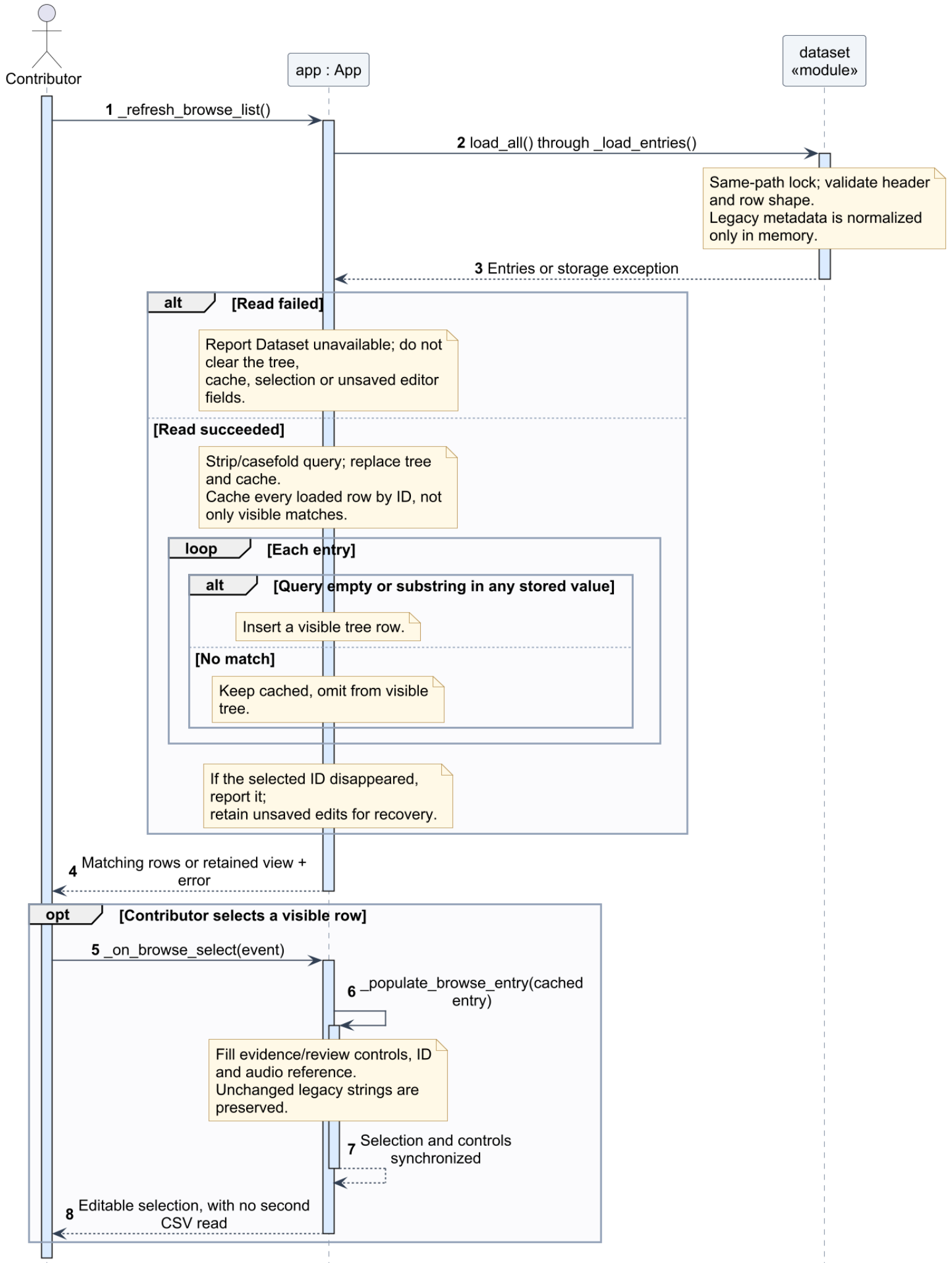
Protocol dispatch is structural; no per-user global CSV path is mutated.
No class association multiplicity is implied by a sequence participant.
Legacy CSV/profile/screenshot access requires no bound adapter and explicit no-auth maintenance.
Default web requests require authentication; production refuses require_auth=False.

Desktop attachment and truthful entry saving



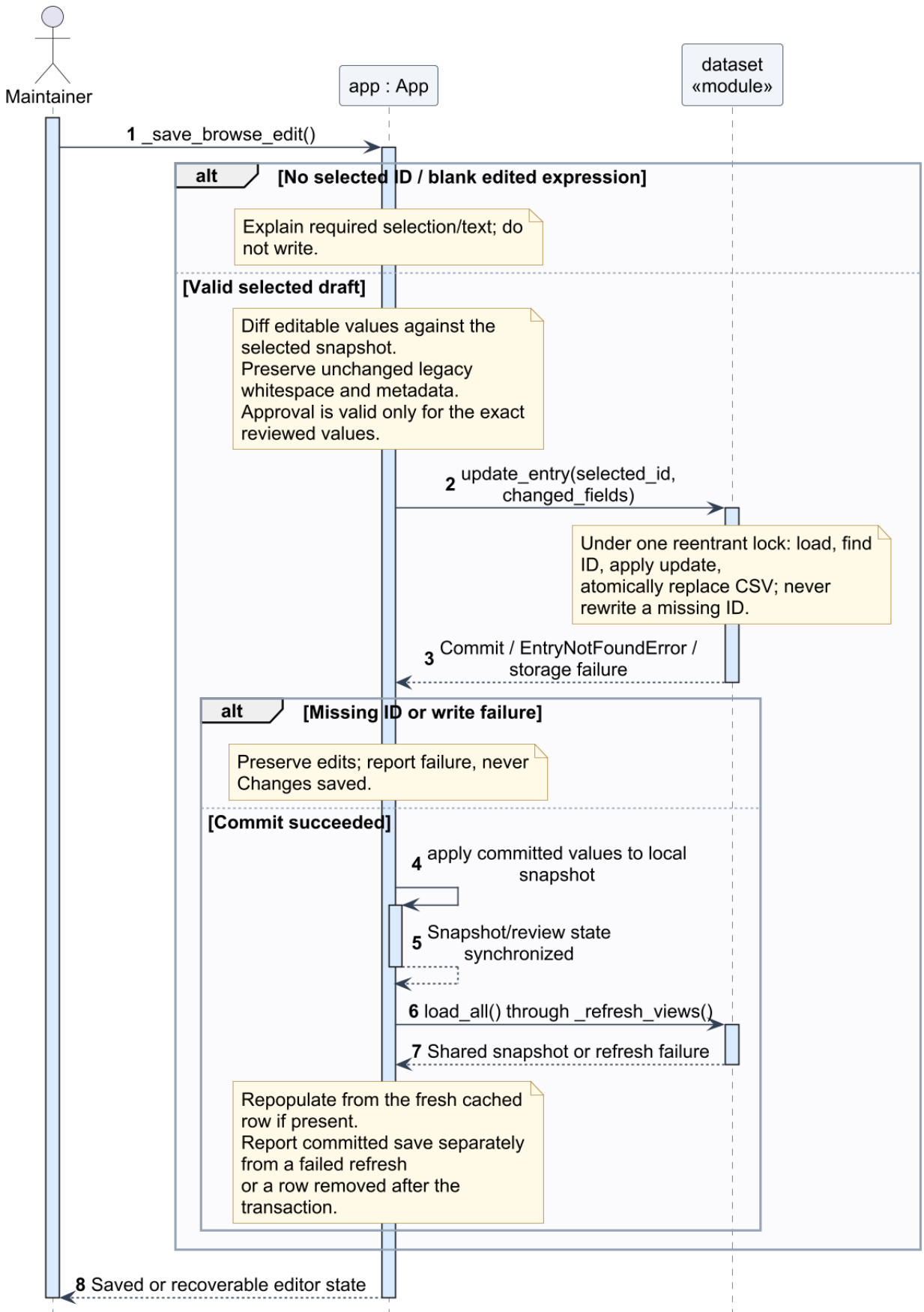
Preserved desktop compatibility only; authenticated web saves never use this global CSV. Files are staged before row creation; CSV and audio are not one transaction. Committed audio survives draft clearing and row deletion. Dialog/widget and low-level I/O calls are condensed, not additional messages.

Desktop search, preserved views and cached selection



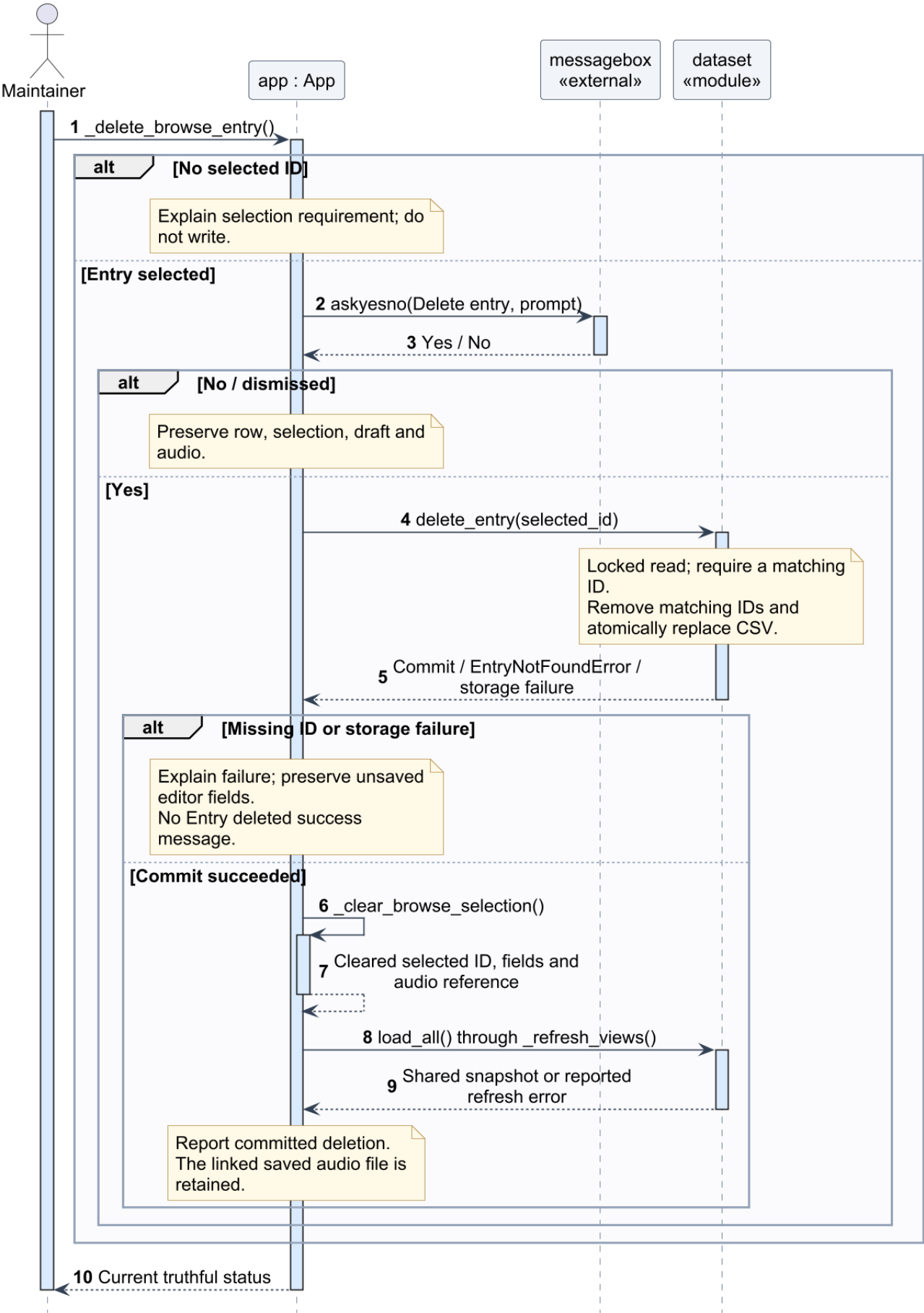
Preserved desktop compatibility only; the web collection reads the selected private project. IDs should remain unique; manually corrupted identifiers are not repaired here. Filtering is read-only. Empty results are distinct from a failed read.

Desktop changed-field editing and review integrity



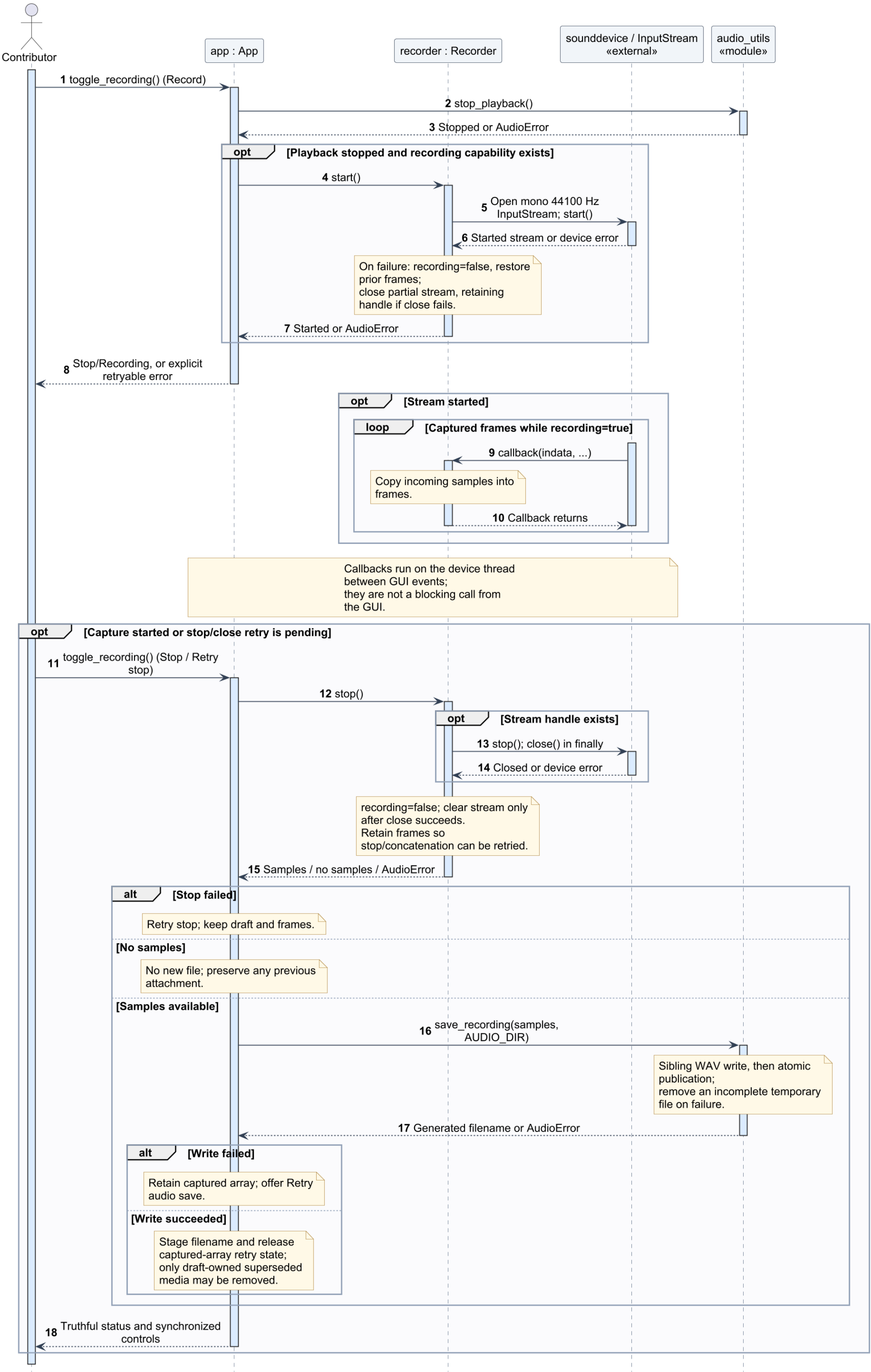
Preserved desktop compatibility only; web mutations also retain private entry revisions. Desktop edits preserve ID, timestamp, contributor and audio reference. Text, glosses, topic, location, notes, language and lexical metadata are editable. Locks prevent interleaved writes, not semantic last-writer-wins conflicts.

Confirmed deletion without false success or media loss



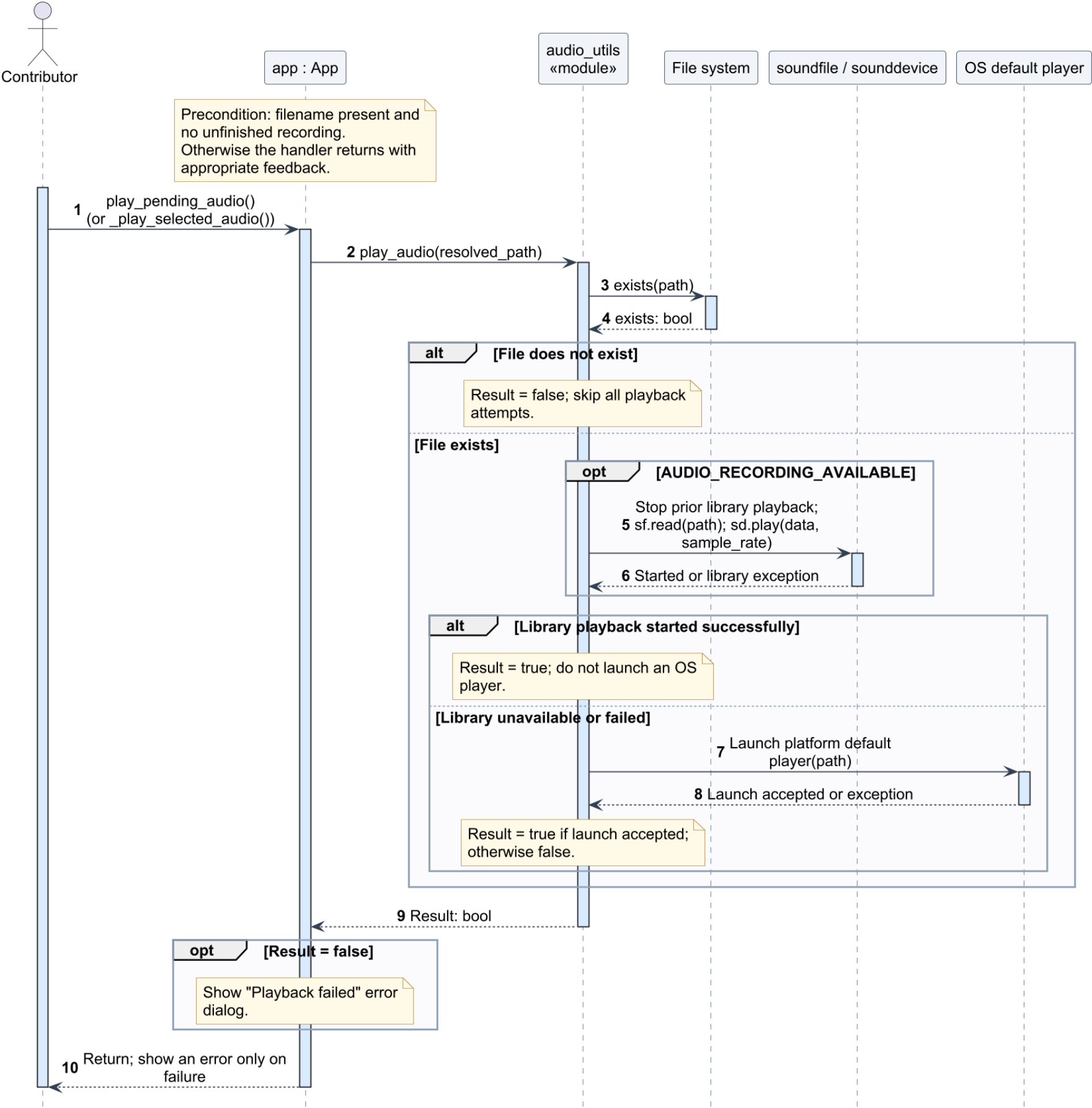
Preserved desktop compatibility only; the authenticated web workspace has separate revisions. Confirmation is a decision, not an authorization or authentication check. Atomic replacement protects write integrity; there is no undo stack.

Retryable microphone capture and WAV staging



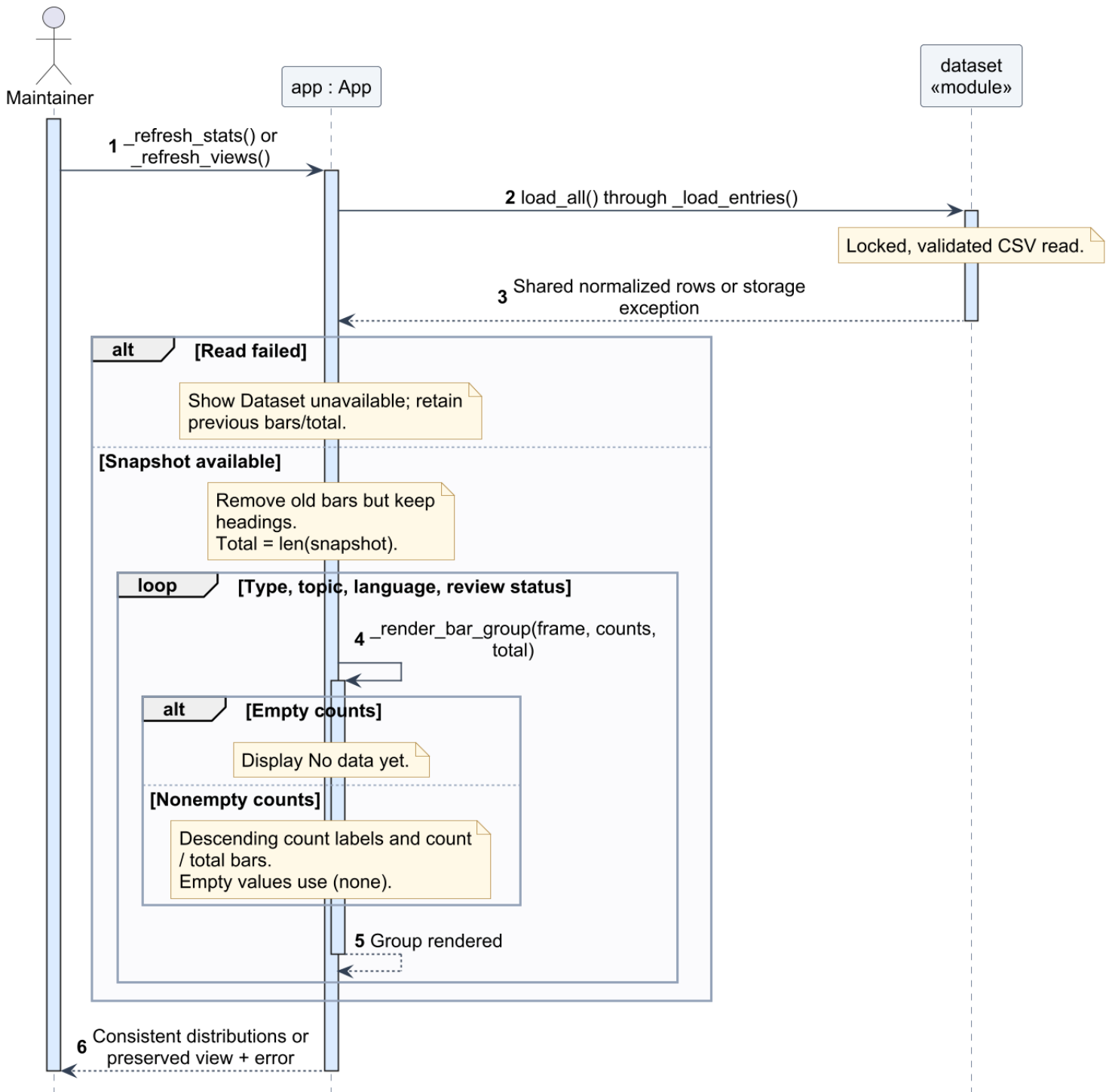
Preserved desktop recorder, not browser dictation or remote transcription.
Device activation spans each callback delivery interval below the start request.
Save, attach and playback are blocked while audio remains unfinished.
Retry audio save calls save_recording again without opening the microphone.
Clear/close deliberately discard draft-owned audio; saved recordings survive.

Preserved desktop playback with library and OS fallback



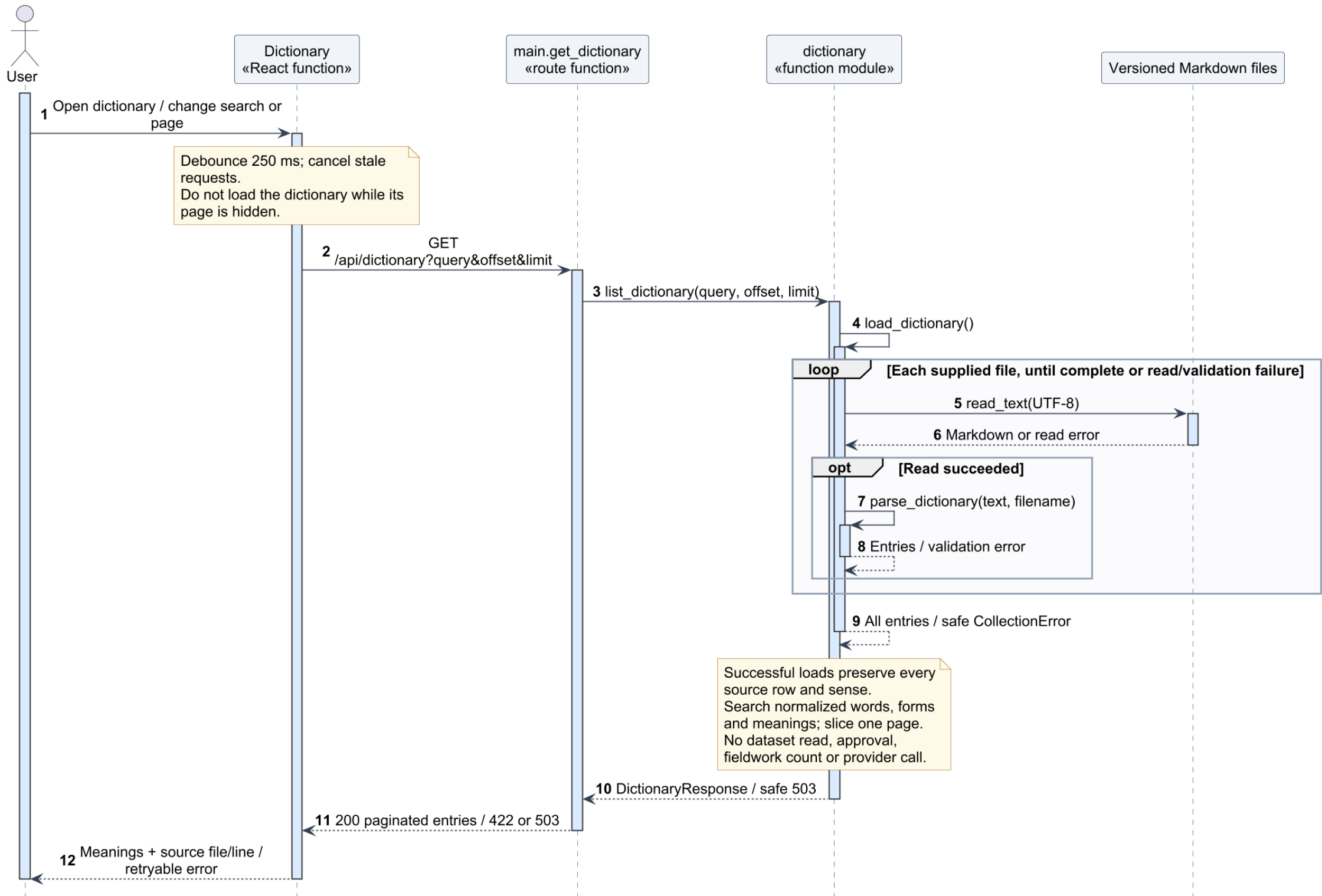
This is the desktop compatibility path, not the browser's private audio download flow. true means a playback request was accepted, not that sound was heard to completion. OS launch acceptance is not playback completion; media decoding is external. Expected audio errors are reported by the GUI; CSV is unchanged. Clear/close can stop in-process playback, not an external OS player's lifetime.

Preserved desktop statistics from one snapshot



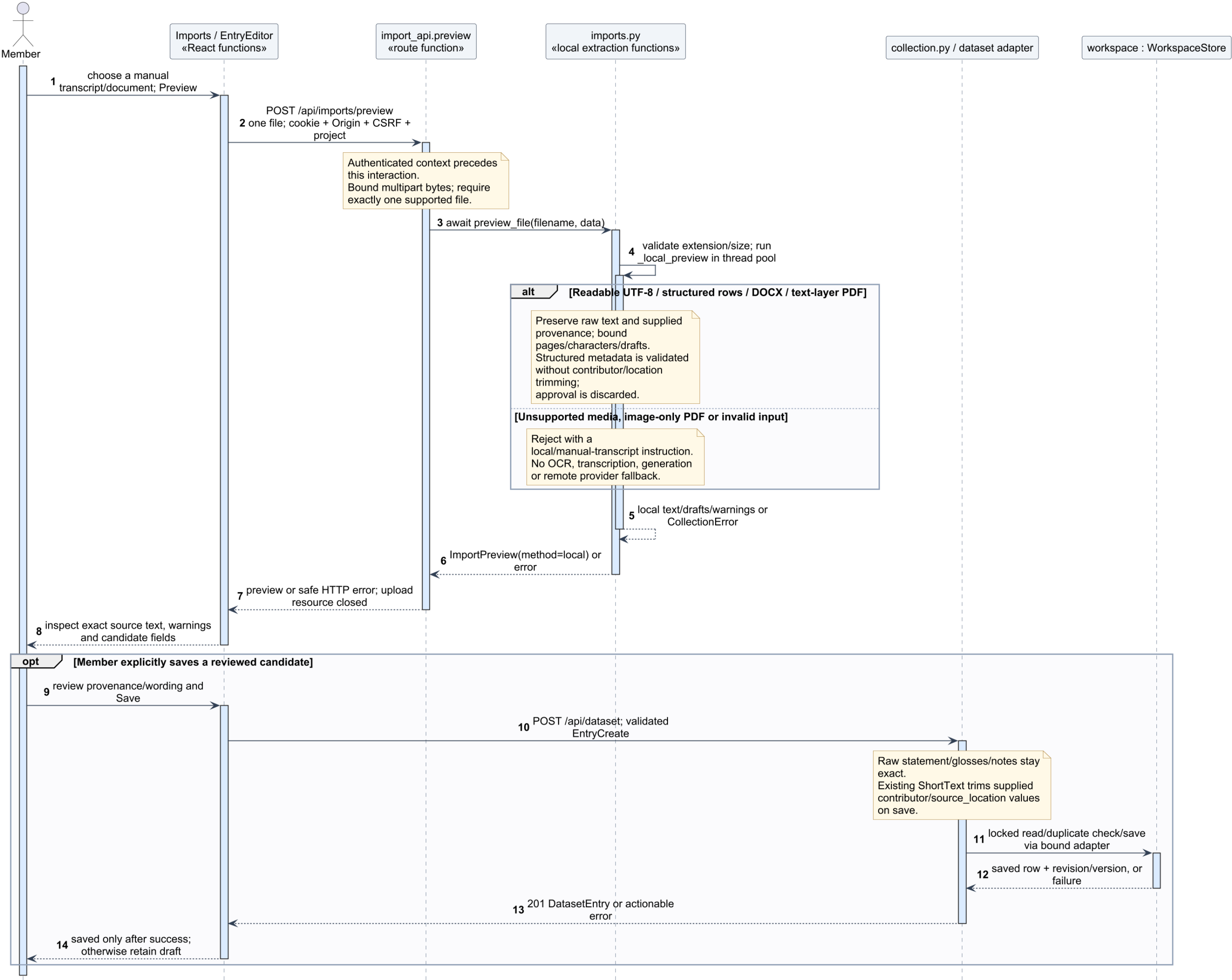
`_refresh_views` also supplies the same rows to count, recent and Browse views.
Direct Stats refresh loads once; `count_by()` is not called by this handler.
Earlier desktop benchmarks are historical; this diagram makes no current timing claim.

Read-only dictionary lookup: source rows, not fieldwork



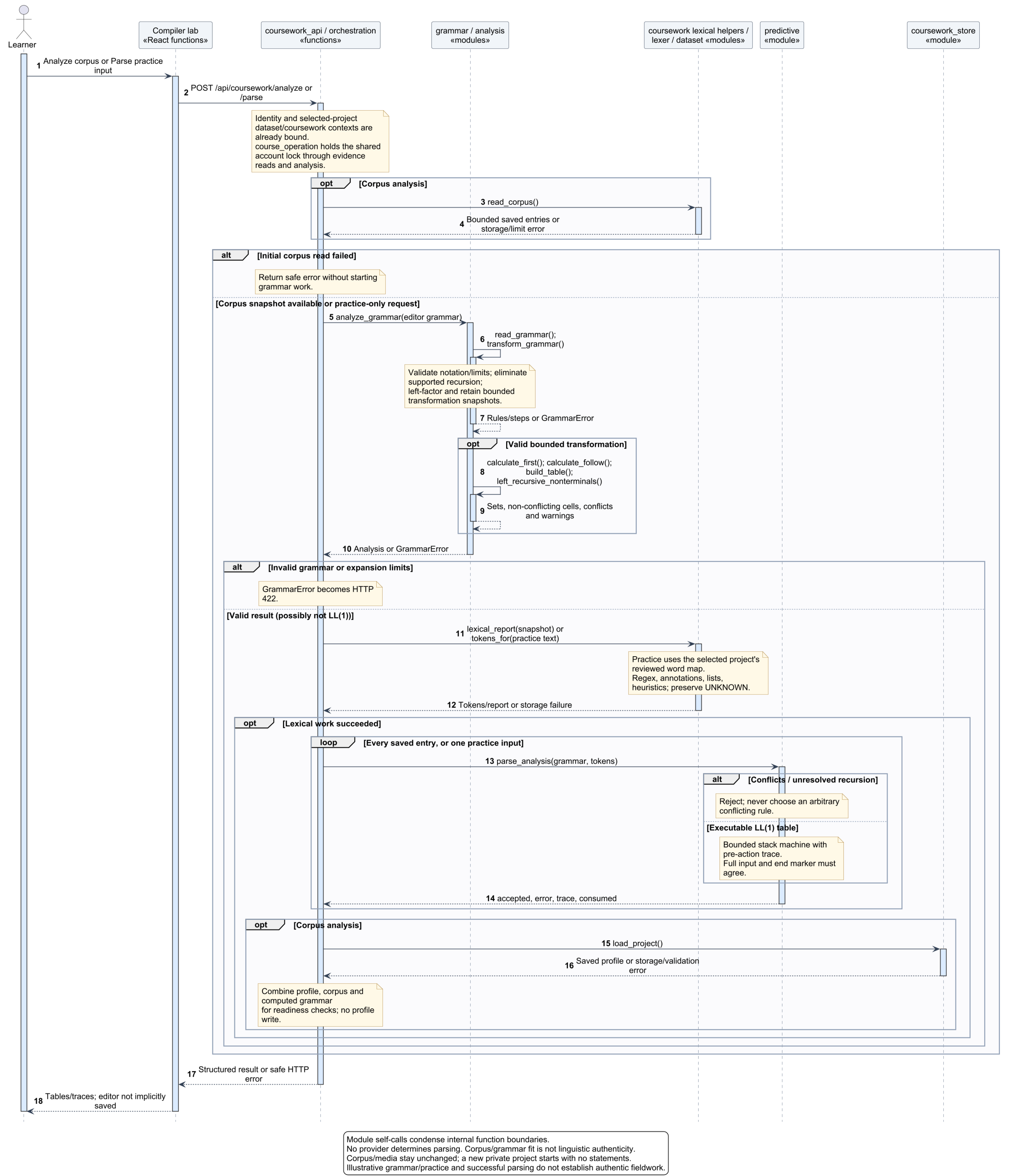
Every dictionary row remains a supplied reference, including repeated forms and senses. English meanings and source origin are not a POS annotation or a reviewed French gloss. The separate practice library is constructed material; neither source is real collected fieldwork. This lookup has no translation/generation side effect.

Local text preview and explicit human-reviewed collection save

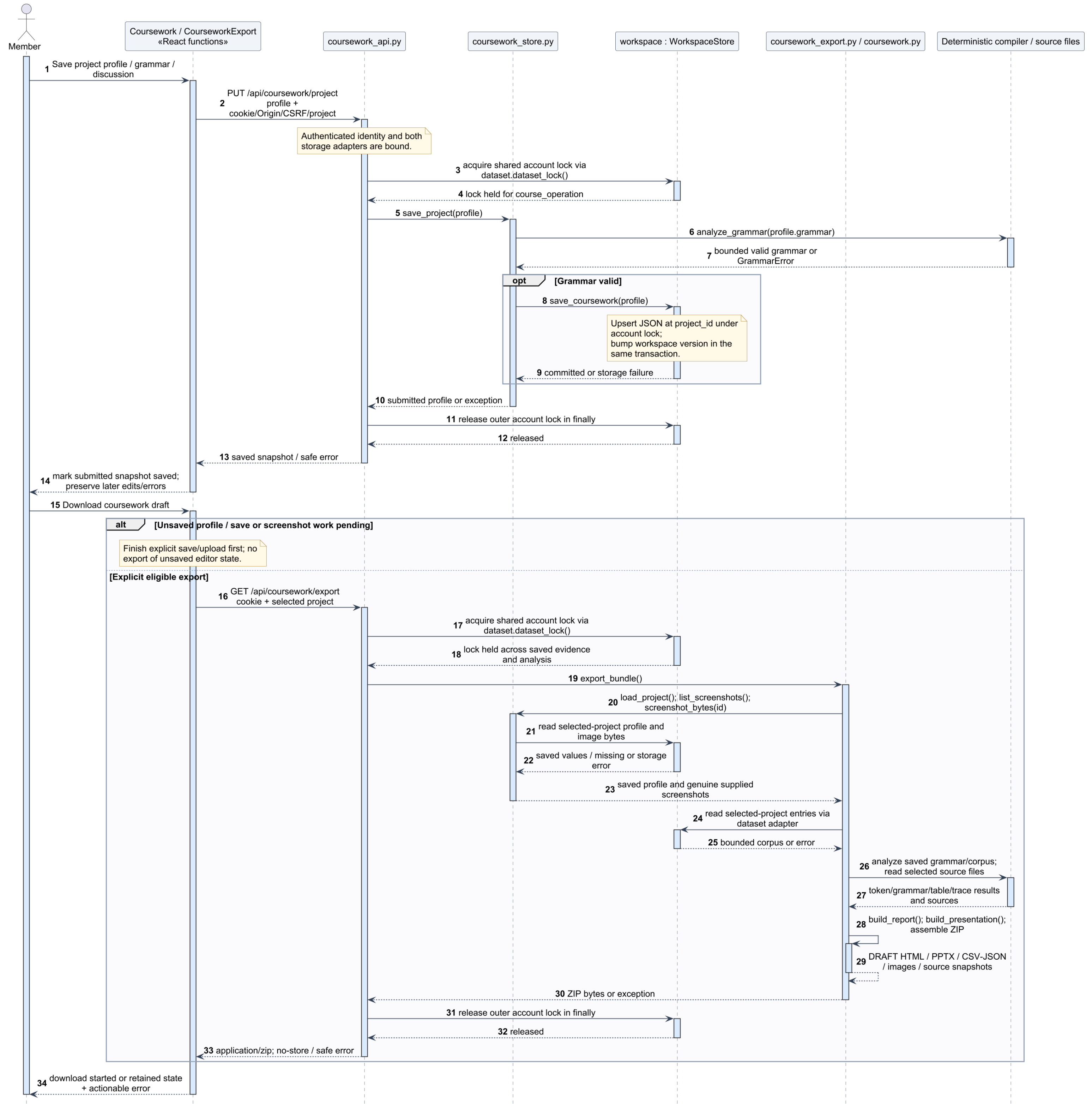


Extraction is local to the application server, not necessarily on-device.
Preview does not save the original document as an attachment or auto-save entries.
Imported text is not proof of real fieldwork; the member supplies authenticity and provenance.
No suggest endpoint or cloud-consent flow remains.

Authenticated compiler lab: grammar analysis and category-token parsing

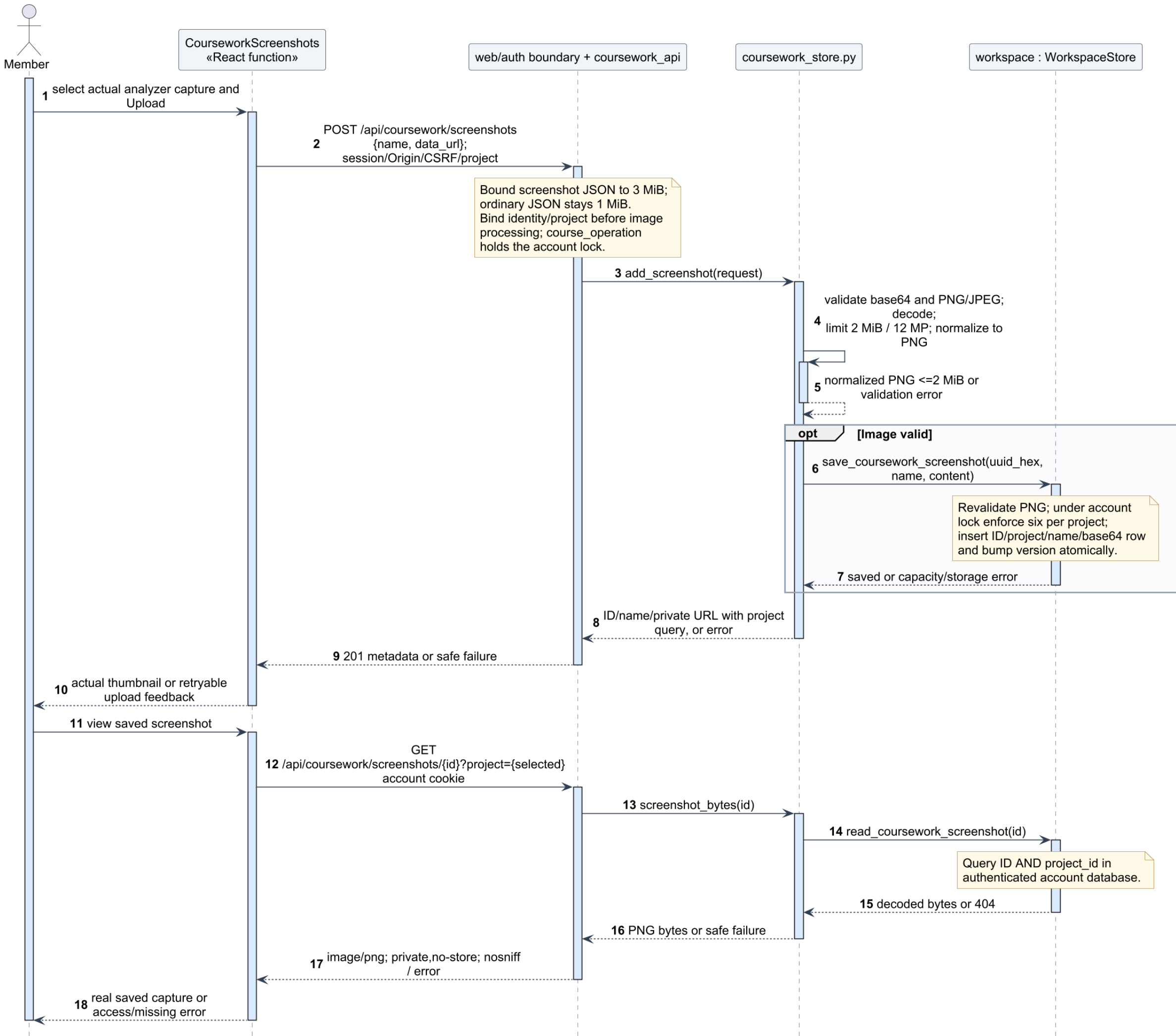


Save project-scoped coursework; export only saved evidence as a DRAFT



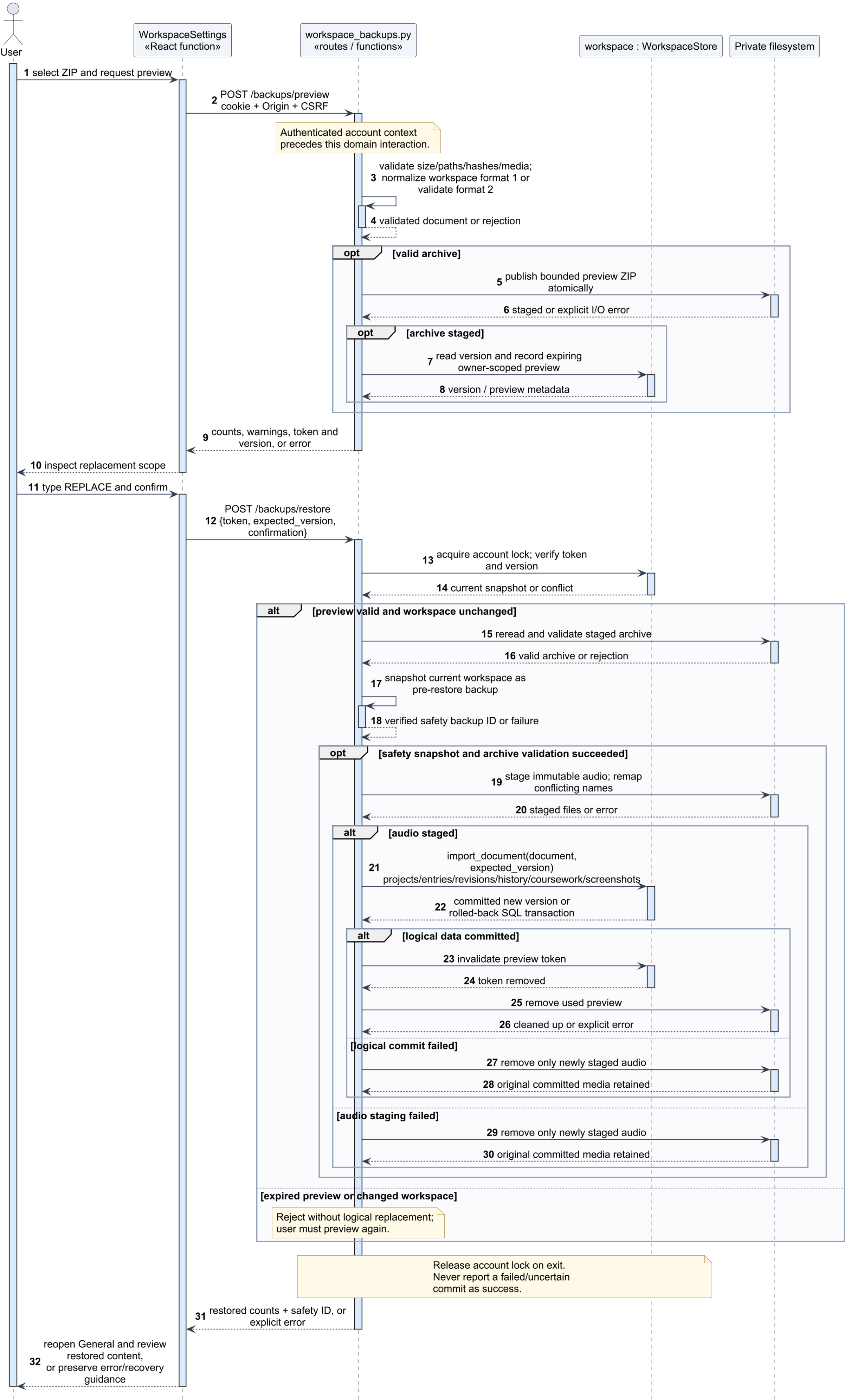
The reentrant account lock prevents profile/corpus/image revision mixing by account mutations. Lock-call activations are not lock ownership duration; ownership spans the whole operation. Source files/external filesystem writers are not part of an account database transaction. Real raw text and contributor metadata are private: review before sharing with course recipients. Raw recordings are not embedded here; full workspace backups retain recordings and coursework. Generated regression expectations replay current parser decisions, not an independent linguistic oracle. Missing research stays visible; 25 sections do not guarantee a 25-30-page final report.

Genuine screenshot evidence: normalized write and owner/project-scoped bytes



A selected project never selects another user's account directory.
Normal requests do not use legacy screenshot paths; stored bytes are not publicly served files.
Upload/delete mutate the workspace version; a prior restore preview becomes stale.
Image validation cannot prove capture authenticity; the student must supply genuine evidence.

Validated account backup restore with a safety copy



Backups replace logical workspace data, not account credentials or server settings. Workspace document format 2 includes profile JSON and screenshot base64; outer manifest stays format 1. Older format-1 documents normalize to empty coursework/screenshots before replacement. Profile/screenshot mutations invalidate a previously captured expected_version. Late client cancellation does not promise server rollback. Existing committed media remains available for recovery; this is not a cross-file ACID transaction.